

**ДЕРЖАВНИЙ УНІВЕРСИТЕТ
ІНФОРМАЦІЙНО-КОМУНІКАЦІЙНИХ ТЕХНОЛОГІЙ
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ
ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ
КАФЕДРА ІНЖЕНЕРІЇ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ**

КУРСОВА РОБОТА

на тему:

«Розробка програмного забезпечення для керування базою даних магазину»

з дисципліни:

«Об'єктно-орієнтоване програмування C#»

студента 2 курсу групи ПД-22

Кафедри інженерії програмного забезпечення

Шеренка Олександра Олександровича

Викладач

асистент кафедри

Інженерії програмного забезпечення,

Ярошевський Олександр Вікторович

оцінка _____

Київ 2026

РЕФЕРАТ

Курсова робота, 60 с., 35 рис., 3 додатки, 12 джерел.

Ключові слова: інтернет-магазин, C#, ASP.NET Core, SQLite, Entity Framework Core, вебзастосунок, база даних, електронна комерція, MVC, CRUD, авторизація, кошик товарів, онлайн-замовлення, автоматизація продажів.

Об'єктом дослідження було: процес розробки вебзастосунку для автоматизації роботи інтернет-магазину та управління товарами, замовленнями і користувачами.

Метою дослідження було: проектування та розробка застосунку інтернет-магазину з використанням мови програмування C#, фреймворку ASP.NET Core та технології Entity Framework Core для роботи з реляційною базою даних SQLite. Основною метою роботи є поглиблення теоретичних знань та отримання практичних навичок у створенні сучасних вебзастосунків, реалізації клієнт-серверної взаємодії, організації роботи з базами даних та побудові архітектури програмного забезпечення.

У ході дослідження було одержано: функціональний вебзастосунок інтернет-магазину, який дозволяє користувачам переглядати товари, виконувати пошук і фільтрацію за категоріями, додавати товари до кошика, оформлювати замовлення та авторизуватися у системі. Реалізовано роботу REST API для обміну даними між клієнтською та серверною частиною застосунку. Створено структуру реляційної бази даних SQLite для зберігання інформації про користувачів, товари, категорії, замовлення та кошик покупця.

ЗМІСТ

ВСТУП	6
1 ХАРАКТЕРИСТИКА ТА АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ.....	8
1.1 Розвиток електронної комерції та перехід до онлайн-торгівлі	8
1.2 Аналіз існуючих рішень	9
1.3 Визначення функціональних вимог.....	11
2 ПРИЗНАЧЕННЯ ТА СТРУКТУРА ПРОГРАМНОГО ПРОДУКТУ.....	13
2.1 Визначення вимог до системи	13
2.1.1 Функціональні та нефункціональні вимоги	13
2.2 Аналіз можливих ризиків проєкту	15
2.3. Цільова аудиторія системи.....	16
2.4 Основні можливості програмного продукту	16
3 МОДЕЛЮВАННЯ ТА ПРОЕКТУВАННЯ ВЕБЗАСТОСУНКУ	17
3.1 Функціональна модель системи.....	17
3.2 Архітектурні рішення вебзастосунку	19
3.2.1 Структура клієнтської та серверної частини	21
3.2.2 Організація взаємодії з базою даних	21
3.3 Проектування бази даних вебзастосунку.....	22
3.3.1 Діаграма послідовності.....	23
3.4 Алгоритм роботи системи	24
4 РОЗРОБКА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ.....	27
4.1 Вибір технологій та інструментів розробки	27
4.2 Реалізація серверної частини застосунку.....	28
4.2.1 Реалізація REST API.....	29
4.3. Реалізація користувацького інтерфейсу.....	30
4.3.1 Головна сторінка та каталог товарів.....	31
4.3.2 Сторінка перегляду товару.....	31

	4
4.3.3 Реалізація кошика покупця.....	32
4.3.4. Адміністративна панель керування	34
4.3.5 Створення замовлення користувачем	35
5 ТЕСТУВАННЯ ТА ПЕРЕВІРКА РОБОТИ СИСТЕМИ	37
5.1 Основи тестування програмного забезпечення.....	37
5.2 Тестування основних сценаріїв роботи користувача	38
5.3. Перевірка роботи бази даних	38
5.4. Результати тестування системи.....	40
ВИСНОВКИ.....	42
ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	44
ДОДАТОК А. ДЕМОНСТРАЦІЙНІ МАТЕРІАЛИ.....	45
ДОДАТОК Б. РЕПОЗИТОРІЙ ПРОГРАМНИХ МОДУЛІВ	54
ДОДАТОК В. ЛІСТИНГИ ОСНОВНИХ ПРОГРАМНИХ МОДУЛІВ.....	55

СЛОВНИК ТЕРМІНІВ

БД	Бази даних
ООП	Об'єктно-орієнтоване програмування
ОС	Операційна система
ПЗ	Програмне забезпечення
CMS	Система керування вмістом (англ. Content Management System)
РСУБД	Реляційна система управління базами даних
SQL	Мова структурованих запитів (англ. Structured Query Language)
C#	Мова програмування
IDE	Інтегроване середовище розробки
HTML	Мова гіпертексту (англ. HyperText Markup Language)
HTTP	Протокол передачі гіпертексту (англ. HyperText Transfer Protocol)
UI	Інтерфейс користувача (англ. User Interface)
UX	Досвід користувача (англ. User Experience)
ER	Сутність-зв'язок (англ. Entity-Relationship)
SQLite	Система керування базами даних
API	Прикладний програмний інтерфейс (англ. Application Programming Interface)
Login	Процес авторизації користувача
Auth	Аутентифікація користувача
Session	Сесія користувача

ВСТУП

У сучасному світі інформаційні технології стрімко інтегруються в усі галузі людської діяльності. Це особливо помітно в торгівлі та сфері обслуговування, де впровадження програмного забезпечення допомагає оптимізувати обробку даних, автоматизувати бізнес-процеси та мінімізувати помилки, зумовлені людським фактором.

Наразі більшість торговельних підприємств переходять на автоматизовані системи управління товарообігом, замовленнями та базою клієнтів. Такі рішення дозволяють ефективно вести облік продукції, відстежувати складські залишки, оперативно опрацьовувати запити, а також налагоджувати злагоджену комунікацію між адміністраторами, менеджерами та покупцями. Ключовим елементом цих систем є бази даних, які гарантують надійне зберігання інформації та забезпечують миттєвий доступ до неї.

У даній курсовій роботі розглядається створення інформаційної системи інтернет-крамниці, розробленої за допомогою мови програмування C# та системи керування базами даних SQLite. Створена система дає змогу оперувати товарами, категоріями, покупцями, замовленнями, кошиком користувача, коментарями.

Завданням цієї курсової роботи є створення функціонального програмного додатку для спрощення процесів інтернет-крамниці. У поточних умовах розвитку електронної торгівлі зростає потреба у швидких, комфортних та стабільних системах, які дозволяють дієво керувати продукцією, замовленнями та даними про клієнтів.

Запропонована система повинна гарантувати зручне сполучення між користувачем та сховищами даних, зменшити ручне опрацювання відомостей та знизити вірогідність хиб під час роботи з великим обсягом інформації. Особливий наголос робиться на легкості та природності інтерфейсу, що дозволяє користувачам оперативно опанувати роботу з програмою без особливої підготовки.

Актуальність даної теми полягає в тому, що зі зростанням електронної комерції збільшується потреба у швидких та надійних механізмах обліку товарів і обробки

замовлень. Автоматизація цих процесів дає змогу підприємствам більш ефективно організовувати свою діяльність, покращувати рівень обслуговування клієнтів та вдосконалювати внутрішні операції.

Створена система може бути використана як навчальний зразок створення настільного чи клієнтського застосунку для керування інтернет-крамницею, а також як початкова модель для подальшого розширення функціоналу та впровадження складніших рішень.

1 ХАРАКТЕРИСТИКА ТА АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ

1.1 Розвиток електронної комерції та перехід до онлайн-торгівлі

Розвиток торгівлі у XXI столітті щільно пов'язаний із цифровими технологіями та інтернетом. Звичайні фізичні крамниці поступово втрачають свою головну роль, оскільки дедалі більше купівель відбувається онлайн. Поява інтернету змінила спосіб придбання та збуту товарів, зробивши процес більш стрімким і комфортним. Тепер покупці можуть оглядати товари й оформлювати замовлення з будь-якого місця, а продавці — взаємодіяти з клієнтами через електронні платформи.

Початок розвитку електронної торгівлі стало можливим завдяки прогресові інтернету та веб-технологій. Перші інтернет-магазини дали змогу споживачам оглядати товари та оформлювати замовлення у мережі без відвідування фізичних крамниць. Це стало можливим завдяки розвитку веб-серверів, систем безпечних інтернет-розрахунків і сховищ даних. Саме сховища даних відіграють важливу роль, оскільки дозволяють стрімко зберігати та опрацьовувати великі обсяги відомостей, гарантуючи точність і надійність функціонування системи.

З розвитком мережі інтернет онлайн-торгівля почала поступово витіснити звичайні крамниці. Головною причиною цього стала комфортність для клієнтів. Споживачі здобули змогу заощаджувати час, не бути залежними від розташування магазину та мати доступ до великого асортименту товарів. Також онлайн-майданчики дозволяють звіряти вартість, читати відгуки інших покупців та оформлювати замовлення у будь-яку добу. Усе це зробило мережеві магазини більш затребуваними та зручними, аніж традиційні форми збуту.

На сьогодні електронна комерція є важливою частиною світової економіки. Онлайн-платформи жваво використовуються як великими компаніями, так і малим бізнесом для продажу товарів і послуг.

Перехід до онлайн-торгівлі став природним етапом розвитку сучасного суспільства та технологій. Водночас успішна робота в цифровому середовищі потребує вправно організованих внутрішніх процесів і надійних програмних рішень.

1.2 Аналіз існуючих рішень

На сьогоднішній день є значна кількість програмних розробок для організації електронної комерції та керування інтернет-крамницями. Подібні системи дають змогу автоматизувати процеси збуту товарів, обліковування клієнтів, опрацювання замовлень та ведення складського обліку.

До найвідоміших онлайн-торговельних майданчиків належать Amazon, eBay, AliExpress, Rozetka, OLX, Walmart та Etsy. Ці сервіси є різними за форматом роботи, але всі вони забезпечують можливість придбання та збуту товарів через інтернет. Наводячи приклад, Amazon та AliExpress сфокусовані на масовому продажу товарів по всьому світові, тоді як eBay додатково підтримує аукціонний формат торгівлі.

Усі ці платформи різняться підходом до організації продажів, але мають спільну мету — забезпечення зручної взаємодії між продавцем і покупцем, автоматизацію процесів замовлення та обробки даних. Нижче наведено таблицю 1.1 з переліком найвідоміших платформ.

Таблиця 1.1

Програми-аналоги

Назва платформ и	Тип	Особливості використання	Основна функція платформи
Amazon	Веб / мобільний	Світовий інтернет-магазин з великим вибором продукції. Володіє системою порад.	Продаж товарів онлайн по всьому світу.

Програми-аналоги

Назва платформи	Тип	Особливості використання	Основна функція платформи
eBay	Веб / мобільний	Платформа для збуту нових та вживаних речей. Аукціонна система, де люди роблять ставки.	Продаж товарів через аукціон і оголошення.
AliExpress	Веб / мобільний	Онлайн-магазин з великою кількістю недорогих товарів. Орієнтована на світову доставку з Китаю.	Роздрібний продаж товарів напряму від виробників.
Rozetka	Веб / мобільний	Найбільший український інтернет-магазин. Продає техніку, одяг та інші товари з гарантією та доставкою по Україні.	Роздрібна онлайн-торгівля.
Walmart	Веб / мобільний / ПК	Велика міжнародна мережа магазинів з онлайн-продажем. Поєднує офлайн і онлайн торгівлю.	Рітейл з онлайн і офлайн продажем.

Аналіз платформ-аналогів засвідчив, що на сучасному ринку функціонує значна кількість онлайн-майданчиків та торговельних платформ, які суттєво відрізняються за своїм функціоналом і призначенням (див. рис. 1.1, див. рис. 1.2). Серед них представлені як великі міжнародні маркетплейси з широким асортиментом товарів, так і локальні інтернет-магазини та сервіси розміщення оголошень.

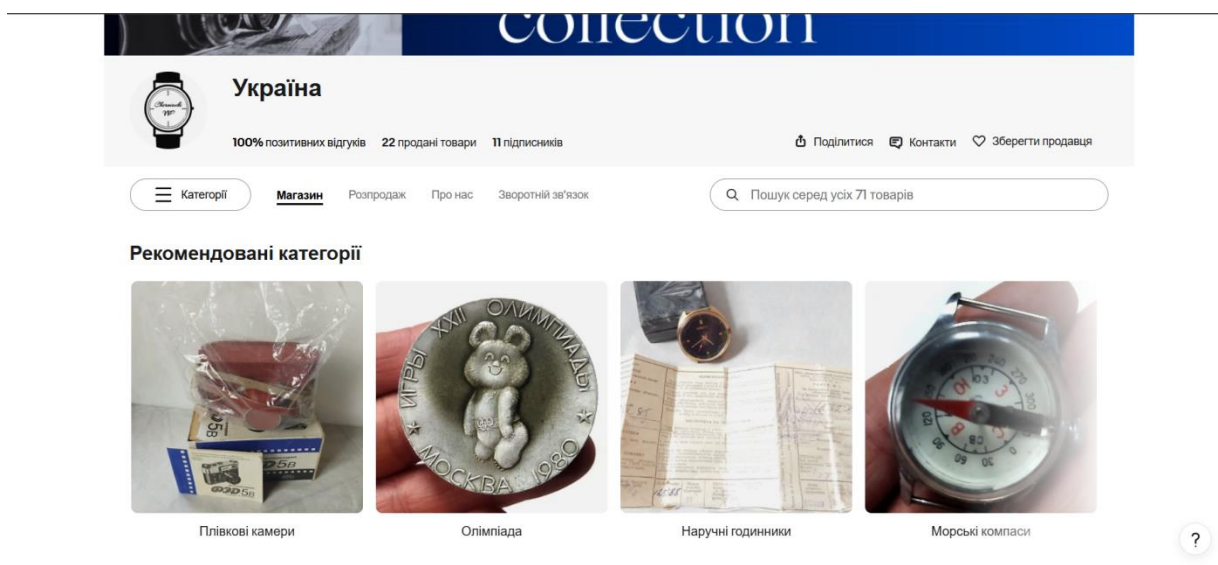


Рис. 1.1 Головна сторінка eBay

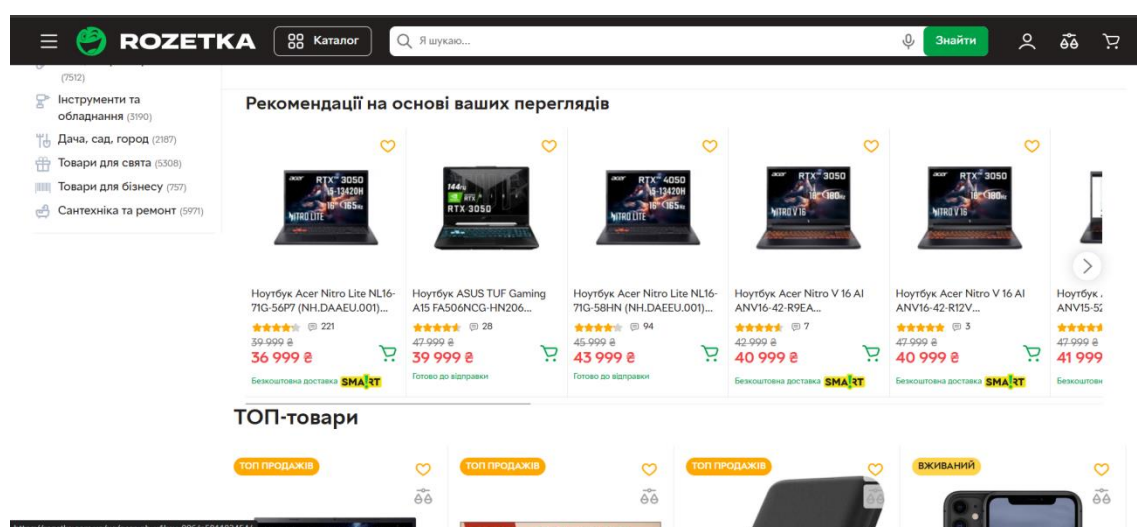


Рис. 1.2 Головна сторінка Rozetka

1.3 Визначення функціональних вимог

Функціональні вимоги окреслюють головні спроможності програмної системи та описують, які завдання вона має виконувати для споживача. Система інтернет-магазину, що розробляється, повинна гарантувати повний цикл роботи з продукцією, клієнтами та замовленнями.

Ключовими функціональними вимогами до системи є:

1. можливість реєстрації та входу користувачів у систему;
2. перегляд продукції з можливістю сортування за категоріями;

3. додавання, зміна та вилучення товарів (для адміністратора);
4. формування замовлень клієнтами;
5. додавання продукції до кошика та їх оформлення;
6. ведення обліку покупців та історії їхніх придбань;
7. облік складських залишків виробів;
8. можливість додавання відгуків до продукції.

Окрім головних функцій, система мусить забезпечувати надійне зберігання відомостей у базі даних та швидкий доступ до них. Також значущою вимогою є зручний та зрозумілий інтерфейс користувача.

Отже, функціональні вимоги формують підґрунтя майбутньої системи та окреслюють її основні спроможності.

2 ПРИЗНАЧЕННЯ ТА СТРУКТУРА ПРОГРАМНОГО ПРОДУКТУ

2.1 Визначення вимог до системи

Визначення вимог до системи є ключовим етапом у процесі розробки програмного забезпечення, оскільки саме на цьому етапі формуються основні функції та правила функціонування майбутнього застосунку. Розроблена інформаційна система для інтернет-магазину має на меті забезпечити зручну взаємодію користувача з каталогом товарів та процесом оформлення замовлення.

Система повинна давати користувачу можливість переглядати асортимент товарів, ознайомлюватися з їх характеристиками, ціною та наявністю. Також передбачена можливість додавання товарів до кошика та подальшого оформлення замовлення. Важливо забезпечити зручну навігацію по категоріях товарів та швидкий пошук необхідної продукції.

Окрему увагу потрібно приділити роботі з даними користувача. Система повинна забезпечувати реєстрацію та авторизацію клієнта, що дозволяє зберігати історію його замовлень та персональні дані. Це підвищує зручність використання та дозволяє формувати індивідуальний досвід роботи з системою.

Крім функціональних можливостей, система повинна забезпечувати стабільну роботу з базою даних, швидко обробку запитів та збереження цілісності інформації. Інтерфейс застосунку має бути інтуїтивно зрозумілим, щоб користувач міг легко виконувати всі необхідні дії без додаткового навчання.

Отже, вимоги до системи визначають базову логіку роботи інтернет-магазину та забезпечують комфортну взаємодію користувача з програмним продуктом.

2.1.1 Функціональні та нефункціональні вимоги

У процесі розробки програмного забезпечення вимоги поділяються на функціональні та нефункціональні. Функціональні вимоги визначають основні можливості системи, тобто те, які дії вона повинна виконувати. Нефункціональні

вимоги описують якість роботи системи, її продуктивність, надійність та зручність використання.

Правильне визначення обох типів вимог є важливим для створення стабільного та зручного програмного продукту, який відповідає потребам користувачів та забезпечує коректну роботу всіх модулів системи.

Функціональні вимоги визначають основний набір операцій, які виконує інтернет-магазин. До них належать робота з товарами, користувачами та замовленнями, а також взаємодія з базою даних.

Функціональні вимоги:

1. реєстрація та авторизація користувачів у системі;
2. перегляд каталогу товарів;
3. перегляд детальної інформації про товар;
4. пошук та фільтрація товарів за категорією, ціною та іншими параметрами;
5. додавання та видалення товарів з кошика;
6. оформлення замовлення з вибором способу доставки;
7. перегляд історії замовлень у особистому кабінеті користувача;
8. додавання коментарів до товару;
9. управління товарами адміністратором (додавання, редагування, видалення).

Нефункціональні вимоги описують якість роботи системи та умови її функціонування. Вони не визначають конкретні дії, але впливають на загальну ефективність та зручність використання програмного продукту.

Нефункціональні вимоги:

1. час відповіді сервера на запит користувача не перевищує 2 секунди при навантаженні до 500 одночасних користувачів;
2. система має підтримувати одночасну роботу не менше 1000 користувачів без зниження продуктивності;
3. коефіцієнт доступності системи (uptime) — не менше 99,5% на місяць;
4. паролі користувачів зберігаються у хешованому вигляді (алгоритм bcrypt);
передача даних здійснюється виключно через HTTPS;

5. інтерфейс має відповідати принципам адаптивного дизайну та коректно відображатись на пристроях із шириною екрана від 320px;

6. архітектура системи має допускати додавання нових модулів без зміни існуючого коду (дотримання принципу Open/Closed);

7. застосунок має коректно працювати у браузерях Chrome, Firefox, Safari та Edge актуальних версій.

2.2 Аналіз можливих ризиків проєкту

Одним із найпоширеніших ризиків є помилки в програмному коді. Такі помилки можуть викликати неправильну роботу окремих функцій, наприклад, авторизації користувачів, додавання товарів до кошика або оформлення замовлення.

Також можливі логічні помилки, які не призводять до аварійного завершення програми, але можуть дати неправильні результати роботи системи.

Іншим важливим ризиком є проблеми з базою даних. Це можуть бути помилки підключення до SQLite, некоректні SQL-запити, порушення зв'язків між таблицями або неправильне зберігання даних. У такому випадку може бути втрачена цілісність інформації або виникнути труднощі з доступом до неї.

Також слід враховувати ризик зниження продуктивності системи при збільшенні обсягу даних. Наприклад, при великій кількості товарів або замовлень може сповільнюватися виконання запитів до бази даних, що впливає на швидкість роботи застосунку в цілому.

Окрему увагу слід приділити помилкам користувача під час взаємодії з системою. Некоректне введення даних або неправильне використання функціоналу може призвести до збоїв у роботі окремих модулів системи або появи помилкової інформації. Наприклад, користувач може ввести неправильний формат електронної пошти, залишити порожні поля при оформленні замовлення або вказати некоректну кількість товару. Такі ситуації можуть впливати на коректність обробки замовлень і загальну роботу системи. Тому важливо передбачити перевірку введених даних та обмеження для уникнення подібних помилок у процесі використання програми.

2.3. Цільова аудиторія системи

Цільова аудиторія розроблюваної інформаційної системи інтернет-магазину складається з користувачів, які роблять покупки онлайн та взаємодіють із системою для перегляду товарів, оформлення замовлень і отримання інформації про продукцію.

Основна увага приділяється звичайним користувачам, які не мають спеціальних технічних знань і потребують простого та зрозумілого інтерфейсу. Система націлена на широкий спектр користувачів різного віку, які використовують інтернет для покупок. Це можуть бути студенти та молодь, а також дорослі, які регулярно купують товари онлайн.

Цільова аудиторія включає:

1. звичайні користувачі;
2. постійні покупці;
3. нові користувачі;
4. інтернет-користувачі різного віку.

2.4 Основні можливості програмного продукту

Розроблена інформаційна система інтернет-магазину створена для автоматизації процесів перегляду товарів, додавання товарів в кошик для замовлення та роботи з даними користувачів. Програмний продукт надає базовий функціонал, необхідний для повноцінної роботи онлайн-магазину та зручної взаємодії користувача з системою.

Система дозволяє користувачу переглядати каталог товарів, отримувати детальну інформацію про кожен товар, а також додавати обрані позиції до кошика. Після цього користувач може оформити замовлення та перевірити його статус.

Також передбачено збереження історії покупок, що дозволяє користувачу відслідковувати свої попередні замовлення. Окремо реалізується робота з обліковими даними користувача, зокрема реєстрація та авторизація, що забезпечує персоналізований доступ до функціоналу системи.

3 МОДЕЛЮВАННЯ ТА ПРОЕКТУВАННЯ ВЕБЗАСТОСУНКУ

3.1 Функціональна модель системи

Функціональна модель системи визначає ключові можливості вебзастосунок, сценарії взаємодії користувачів із платформою та перелік функцій, що забезпечують її повноцінну роботу. Розроблений продукт — це інтернет-магазин, функціонал якого охоплює перегляд товарів, керування кошиком, оформлення замовлень та адміністрування контенту.

Головна мета системи полягає в організації зручної взаємодії між користувачем і електронним каталогом через вебінтерфейс. Система забезпечує автоматизацію таких процесів, як перегляд асортименту, пошук потрібних позицій, формування кошика та подальше оформлення замовлення.

У системі передбачено дві основні ролі користувачів:

1. звичайний користувач;
2. адміністратор.

Таблиця 3.1

Можливості Користувача/Адміністратора

Користувач	Адміністратор
Переглядає каталог товарів	Додає нові товари
Виконує пошук товарів	Редагує інформацію про товари
Переглядає інформацію про товар	Видаляє товари
Додає/видаляє товари до кошика	Здійснює логін через свою персональну пошту

Можливості Користувача/Адміністратора

Користувач	Адміністратор
Оформляє замовлення з кошика	Модерує відгуки та коментарі
Залишає коментарі та відгуки	Керує категоріями
Читає опис до товару	Оброблює замовлення

Кожна сутність наділена набором характеристик, необхідних для коректного функціонування системи. Зокрема, сутність «Товар» містить такі атрибути, як найменування, опис, вартість, зображення та приналежність до певної категорії; сутність «Користувач» передбачає зберігання облікових даних, персональної інформації та параметрів, необхідних для ефективної взаємодії з платформою. Окрім того, архітектура системи включає сутності «Коментар», «Категорія» та «Елемент кошика», що забезпечують цілісність функціоналу вебзастосунку та належну взаємодію між його окремими компонентами.

Для візуалізації взаємодії користувачів та адміністратора із системою застосовується UML-діаграма прецедентів (Use Case Diagram), яка демонструє ключові функціональні можливості вебзастосунку та відповідну рольову модель користувачів.

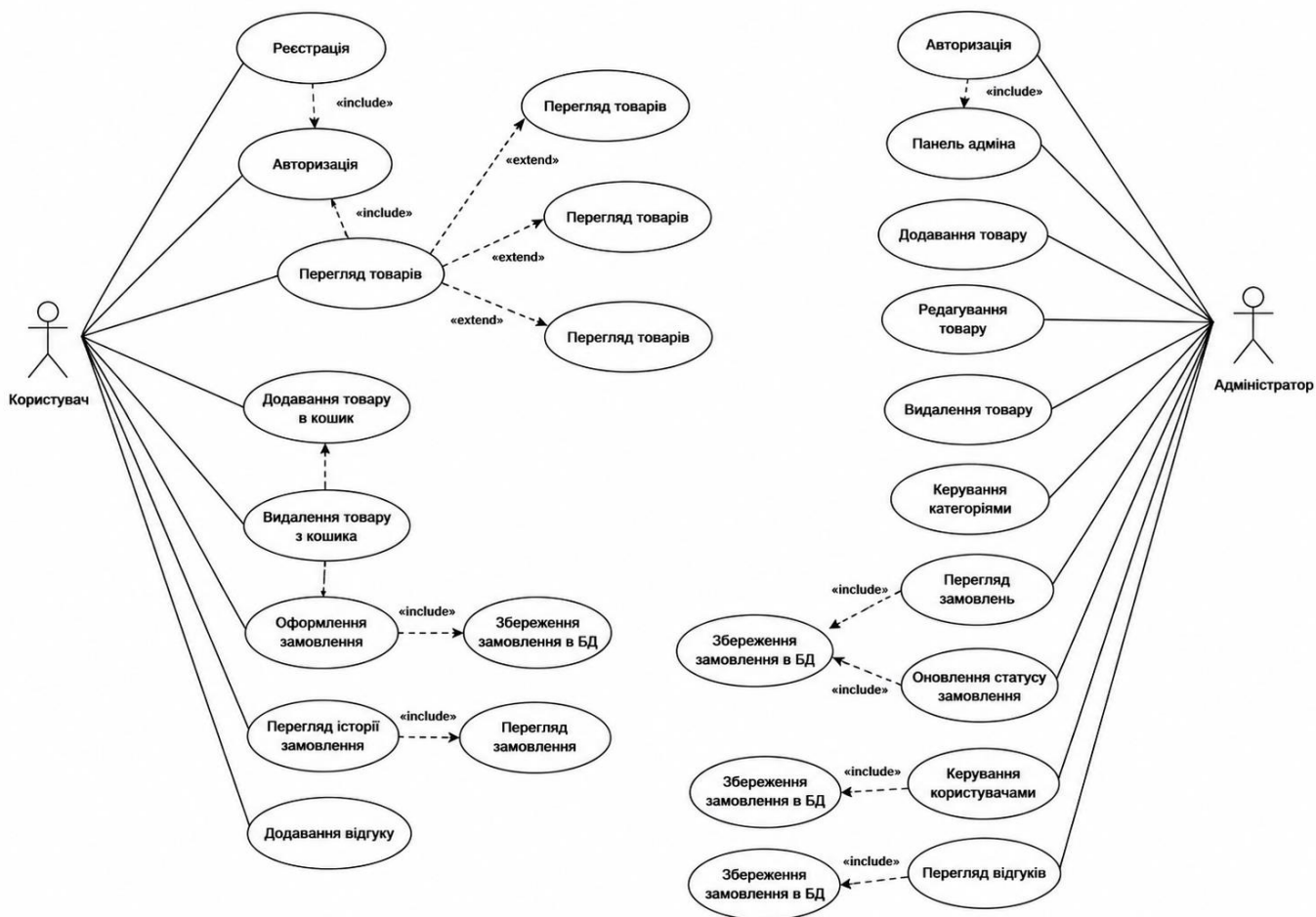


Рис. 3.1 Діаграма прецедентів взаємодії користувача та адміністратора з інформаційною системою

3.2 Архітектурні рішення вебзастосунку

Під час проектування вебзастосунку ключовим етапом є визначення архітектури програмного продукту та оптимізація взаємодії між його структурними компонентами. Виважений вибір архітектурного рішення забезпечує зручність подальшого супроводу коду, належну масштабованість системи, а також сприяє оптимізації процесу розробки та підвищенню надійності функціонування вебресурсу.

Для реалізації функціонала інтернет-магазину було обрано архітектурний шаблон MVC (Model–View–Controller), що інтегрований у фреймворк ASP.NET Core. Зазначена архітектура уможлиблює логічну диференціацію застосунку на автономні компоненти, кожен із яких виконує визначену цільову функцію.

Архітектура вебзастосунку складається з трьох основних частин:

1. Model — відповідає за роботу з даними та логіку програми;
2. View — забезпечує відображення інформації користувачу;
3. Controller — обробляє дії користувача та виконує взаємодію між моделями і представленнями.

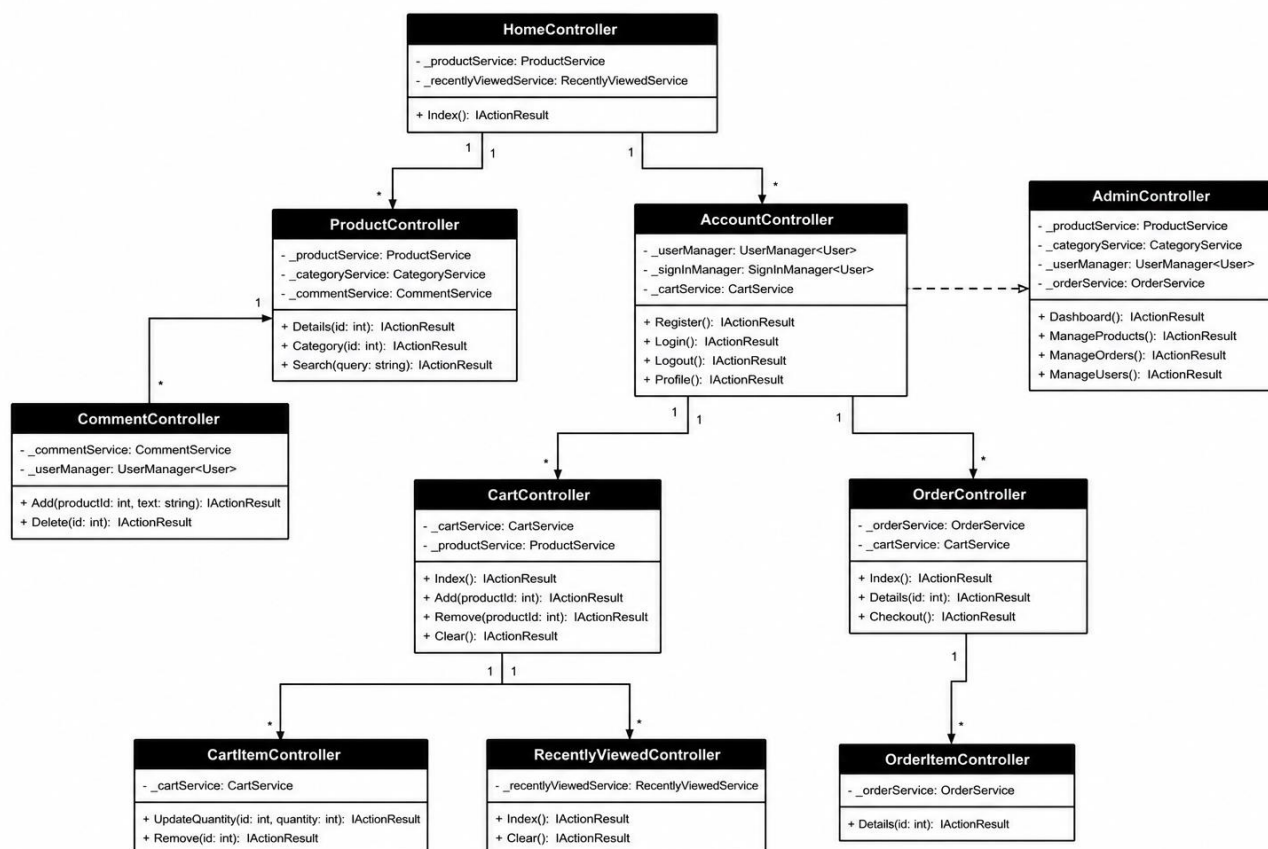


Рис. 3.2 Діаграма класів контролерів вебзастосунку

У процесі роботи вебзастосунку користувач через веббраузер надсилає HTTP-запит до сервера. Контролер отримує цей запит, виконує необхідну обробку та звертається до моделей для отримання або зміни даних. Після цього результати передаються до представлення, яке формує сторінку для користувача. Структура програмного проекту складається з окремих директорій, кожна з яких відповідає за певну частину функціоналу системи.

Основні компоненти проекту

1. Controllers — містять контролери для обробки запитів користувачів;
2. Models — містять моделі даних та класи сутностей;
3. Views — містять сторінки користувацького інтерфейсу;

4. Services — реалізують допоміжну бізнес-логіку;
5. Data — забезпечує роботу з базою даних;
6. ViewModels — використовуються для передачі даних між контролерами та представленнями;
7. wwwroot — містить статичні файли вебзастосунку.

У розробленому проєкті реалізовано декілька контролерів, які відповідають за окремі функціональні можливості системи. Наприклад, `ProductController` забезпечує роботу з товарами, `CartController` відповідає за кошик користувача, `AccountController` реалізує авторизацію та реєстрацію, а `AdminController` використовується для адміністрування системи.

3.2.1 Структура клієнтської та серверної частини

Інтернет-магазин працює за принципом "клієнт-сервер". Це означає, що програма розділена на дві основні частини. Одна частина – це те, що ви бачите і з чим взаємодієте в браузері. Вона дозволяє вам переглядати товари, шукати потрібне, додавати до кошика та робити замовлення. Інша частина – це "мозок" магазину, який працює на сервері (сервер). Він отримує ваші запити, обробляє їх, виконує всю необхідну логіку та взаємодіє з базою даних, де зберігається інформація про товари та замовлення. Зв'язок між цими частинами відбувається за допомогою стандартних інтернет-протоколів (HTTP) та спеціального інтерфейсу (REST API).

Серверна частина побудована на технології ASP.NET Core. Вона керує всіма процесами: від отримання ваших дій до оновлення даних у базі даних SQLite. Для цього використовуються спеціальні компоненти: контролери (які приймають ваші команди), сервіси (які виконують основні операції) та моделі (які описують дані). Такий підхід робить програму більш організованою, легкою для розуміння та подальшого розвитку.

3.2.2 Організація взаємодії з базою даних

Для управління даними вебзастосунку використовується реляційна база даних SQLite. Вона надійно зберігає інформацію про асортимент товарів, облікові записи

користувачів, їхні категорії, вміст кошиків та історію замовлень. SQLite була обрана як оптимальне рішення завдяки своїй простоті, швидкому розгортанню та бездоганній сумісності з ASP.NET Core.

Взаємодія з базою даних здійснюється через потужний інструмент Entity Framework Core. Основний компонент для роботи з даними – клас DbContext, який забезпечує зв'язок з базою та дозволяє виконувати всі необхідні операції з даними (створення, читання, оновлення, видалення).

У базі даних передбачені спеціалізовані таблиці для зберігання деталей про користувачів, товари, категорії, замовлення, кошики та інші ключові елементи системи. Маніпуляції з даними відбуваються шляхом виконання запитів LINQ та виклику відповідних методів сервісів.

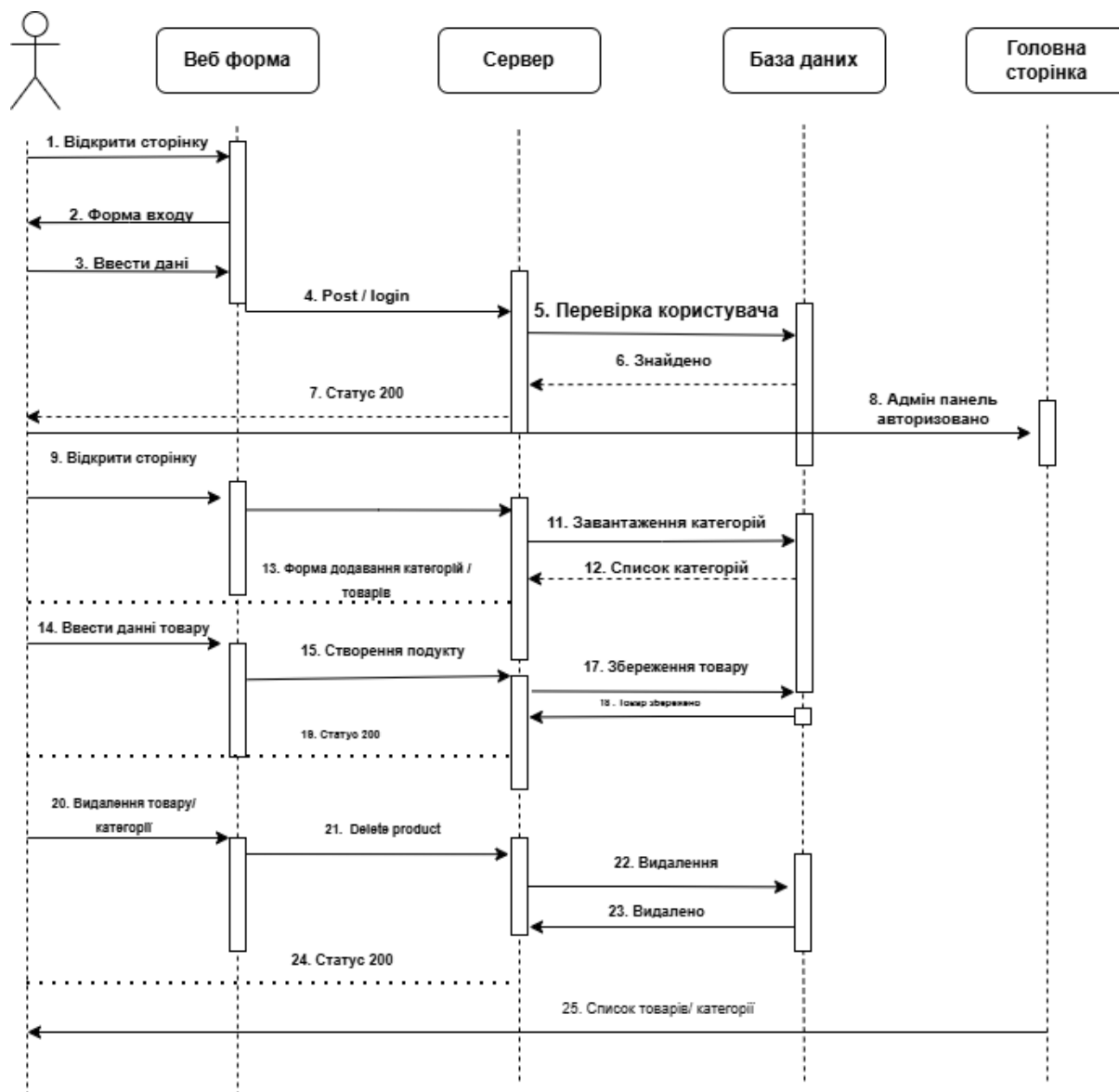
3.3 Проектування бази даних вебзастосунку

База даних становить один із ключових елементів застосунку, оскільки саме вона відповідає за зберігання, обробку та встановлення зв'язків між усіма даними системи. При проектуванні бази даних було враховано специфіку роботи інтернет-магазину, вимогу до швидкого доступу до інформації та потенційну потребу в подальшому розширенні функціоналу.

У створеному вебзастосунку база даних застосовується для фіксації даних про товари, користувачів, категорії, коментарі, елементи кошика та переглянуті товари. Дані між собою взаємопов'язані, що гарантує коректне функціонування системи та взаємодію окремих модулів.

Для збереження даних у вебзастосунку застосовується база даних SQLite. Вона відрізняється малою вагою та компактністю, не вимагає розгортання окремого серверного ПЗ і зберігає всі дані в одному локальному файлі. Використання SQLite є виправданим для невеликих застосунків і навчальних проектів завдяки простоті конфігурації, швидкодії та легкості інтеграції з ASP.NET Core. SQLite надає підтримку базових можливостей реляційних баз даних, зокрема таблиць, зв'язків між сутностями та виконання SQL-запитів. Це дозволяє організувати зберігання

внесення відповідних відомостей дані надсилаються на сервер. Контролер здійснює валідацію введеної інформації та передає її на рівень бізнес-логіки. Сервісний рівень



забезпечує опрацювання запиту та збереження запису в базі даних SQLite. Після успішного завершення операції система генерує відповідне сповіщення та актуалізує перелік товарів(див. рис. 3.4).

Рис. 3.4 Діаграма послідовності взаємодії компонентів

3.4 Алгоритм роботи системи

Алгоритм функціонування вебзастосунку інтернет-магазину визначає, як користувач взаємодіє із системою, і структурує основні процеси програми. Його ключове завдання — забезпечити стабільність роботи платформи, оперативну обробку запитів і коректну взаємодію з базою даних. Робота системи розпочинається з того, що користувач відкриває вебсайт у браузері. Після завантаження домашньої сторінки автоматично надсилається запит до серверної частини для отримання даних про продукти та категорії. Сервер обробляє ці дані та вивантажує їх у клієнтський інтерфейс у форматі JSON, після чого вони відображаються у вебзастосунку. Користувач може переглядати каталог товарів, шукати за ключовими словами, використовувати фільтри для сортування за категоріями або переглядати деталі обраного продукту. При виконанні пошуку чи виборі категорії клієнтська частина генерує HTTP-запит до API.

Сервер обробляє цей запит і надсилає відповідний набір результатів назад. Під час реєстрації нового акаунта система перевіряє коректність введених даних. Якщо все введено правильно, створюється новий запис у базі даних. Для авторизації користувач зазначає електронну пошту та пароль, які перевіряються системою. У разі успішної перевірки відкривається доступ до функцій особистого кабінету. Залишаючись авторизованим, користувач може додавати товари до кошика. Система перевіряє доступність товару на складі та створює відповідний запис у таблиці кошика. Можна змінювати кількість вибраного товару або повністю видаляти його з кошика. Всі зміни миттєво синхронізуються з базою даних і моментально оновлюються на вебсторінці.

Для адміністрування вебзастосунку передбачено функції керування товарами. Адміністратор або менеджер може додавати нові позиції у каталог, вносити зміни до існуючої інформації чи видаляти товари. Усі зміни відразу зберігаються у базі даних SQLite та стають доступними для звичайних користувачів. Серверна частина застосунку взаємодіє з базою даних через Entity Framework Core, використовуючи моделі даних, сервіси та контролери для виконання операцій. Такий підхід спрощує архітектуру програми, робить код більш читабельним і полегшує масштабування функціоналу. У підсумку алгоритм забезпечує узгоджену роботу клієнтської і

серверної частин платформи, стабільну обробку інформації та комфортну взаємодію користувача із системою інтернет-магазину.

4 РОЗРОБКА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.1 Вибір технологій та інструментів розробки

Для розробки вебзастосунку інтернет-магазину було обрано сучасний технологічний стек, що гарантує високу продуктивність, надійність, а також зручність подальшого супроводу та масштабування системи.

Основними інструментами розробки виступає фреймворк ASP.NET Core MVC. Дана технологія уможливорює створення багаторівневих вебзастосунків згідно з архітектурним шаблоном Model-View-Controller (MVC), що забезпечує чітку структурування логіки обробки даних, інтерфейсу користувача та керування запитами. Застосування ASP.NET Core дозволяє створювати кросплатформні системи з оптимізованими показниками швидкодії та ефективним використанням обчислювальних ресурсів.

Як мову програмування обрано C#, що є ключовою мовою платформи .NET. Вона вирізняється лаконічним синтаксисом, підтримкою принципів об'єктно-орієнтованого програмування та наданням розгалуженої екосистеми бібліотек для реалізації серверної логіки будь-якого рівня складності.

Системою управління базами даних обрали SQLite. Ця СУБД легка, компактна і не вимагає установки окремого серверного програмного забезпечення. База даних зберігається в одному файлі, що значно полегшує розгортання та тестування програми. SQLite має хорошу продуктивність для малих і середніх вебпроектів, тому є відмінним вибором для створення інтернет-магазину.

Для створення користувацького інтерфейсу використовують HTML та CSS. HTML визначає, як виглядають сторінки в інтернеті, CSS відповідає за їхнє оформлення та адаптацію до різних пристроїв.

У процесі роботи використовували інтегровану систему розробки Microsoft Visual Studio. Вона дає можливість писати, налагоджувати і тестувати програмний код, а також допомагає автоматично керувати залежностями проекту.

Для написання пояснювальної записки та оформлення текстової частини курсової роботи я користувався текстовим редактором Microsoft Word. Це програма, яка допомагає зручно формувати документи відповідно до необхідних стандартів. Щоб контролювати версії програмного коду і зберігати історію змін, я використовував систему Git. Вона дозволяє ефективно управляти процесом розробки і спрощує підтримку програмного забезпечення.

Вибраний набір технологій гарантує ефективне виконання всіх функцій інтернет-магазину. Це включає каталог товарів, систему категорій, корзину покупця, коментарі до товарів, перегляд нещодавно переглянутих товарів та адміністративну панель для управління контентом.

4.2 Реалізація серверної частини застосунку

Серверна частина вебзастосунку створена з використанням платформи ASP.NET Core MVC. Вона відповідає за обробку запитів від користувачів, взаємодію з базою даних, виконання бізнес-логіки та підготовку відповідей для клієнтської частини.

Основними частинами серверної частини є контролери, моделі даних і спосіб доступу до бази даних через Entity Framework Core. Контролери отримують HTTP-запити від користувачів і вирішують, які дії потрібно виконати для їх обробки. Після того, як контролер отримує запит, він звертається до моделей і контексту бази даних, щоб отримати або змінити інформацію.

Для зберігання даних використовується база даних SQLite. Доступ до неї здійснюється через клас ApplicationDbContext, який забезпечує взаємодію між об'єктами програми та таблицями бази даних. Це дозволяє працювати з даними як з об'єктами C#, без потреби писати багато SQL-запитів(див. рис. 4.1).

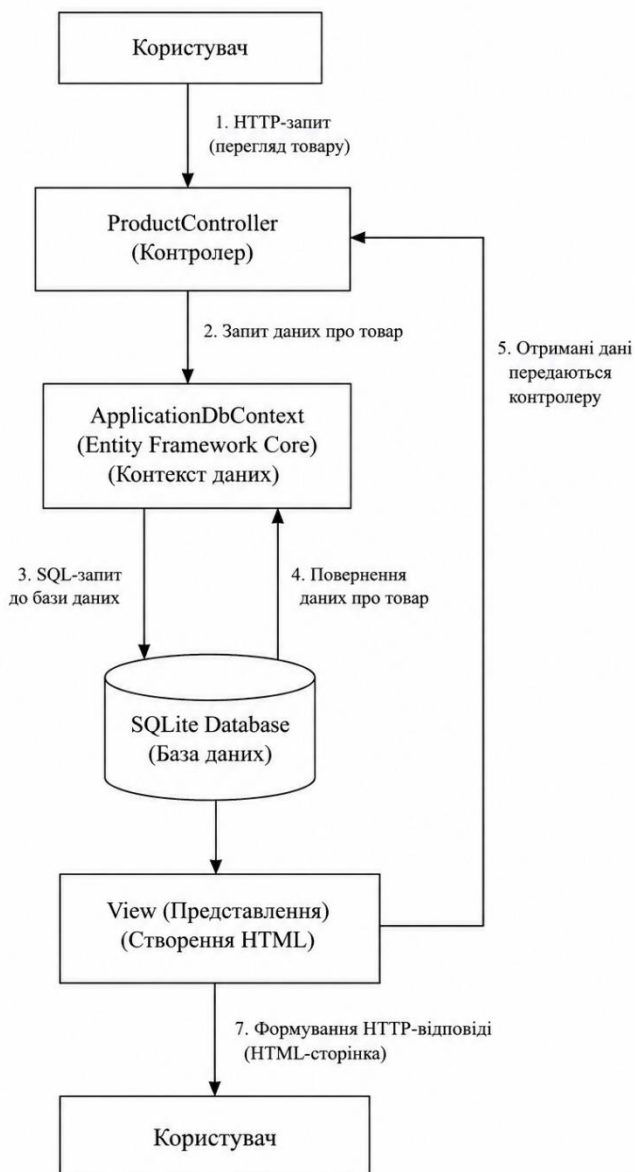


Рис 4.1 Діаграма обробки запиту на перегляд товару

4.2.1 Реалізація REST API

Під час створення вебзастосунку було використано архітектурний патерн MVC (Model-View-Controller) на базі технології ASP.NET Core, що забезпечує чіткий розподіл між логікою, даними та відображенням інформації.

Запити обробляються контролерами, які отримують дані від користувача, виконують необхідні дії та повертають готові HTML-сторінки. Для роботи з базою даних використовується Entity Framework Core, який дозволяє взаємодіяти з SQLite через об'єктну модель без написання SQL-запитів вручну.

Застосунок надає можливість переглядати товари та категорії, додавати їх до кошика, оформлювати замовлення, а також виконувати адміністративні дії — створення, редагування та видалення товарів і категорій. Доступ до адміністративних функцій обмежено роллю Admin за допомогою механізму авторизації ASP.NET Core Identity.

Перед обробкою запиту виконується перевірка вхідних даних за допомогою анотацій валідації моделей, що підвищує надійність і безпеку застосунку. Обмін даними між браузером і сервером відбувається через стандартні HTTP-запити (GET, POST), а сервер у відповідь повертає відрендерені HTML-сторінки на основі шаблонів Razor Views.

Використання патерну MVC разом із Entity Framework Core забезпечує зручну підтримку коду, його масштабованість та можливість подальшого розширення функціональності застосунку.

4.3. Реалізація користувацького інтерфейсу

Користувацький інтерфейс вебзастосунку створений з використанням технологій HTML, CSS та JavaScript. HTML використовується для формування структури вебсторінок і розміщення елементів інтерфейсу. CSS надає можливість оформлювати сторінки, стилізувати елементи та налаштовувати інтерфейс для різних розмірів екранів. JavaScript використовується для створення інтерактивних елементів і поліпшення взаємодії користувачів із системою.

Сторінки створюються та показуються на сервері з використанням технології ASP.NET Core MVC та мови C#. Контролери відповідають на запити користувачів, отримують потрібні дані з бази даних і передають їх до представлень (Views), де формується фінальний інтерфейс вебзастосунку.

В результаті було створено набір сторінок, які дозволяють переглядати каталог товарів, отримувати детальну інформацію про них, працювати з кошиком покупця, входити в систему користувачам та керувати вмістом системи. Нижче будуть подані основні елементи інтерфейсу розробленого вебзастосунку.

4.3.1 Головна сторінка та каталог товарів

Головна сторінка вебзастосунку є основною точкою взаємодії користувача із системою. На ній розміщено каталог товарів, який містить перелік доступної продукції із зазначенням основної інформації. Для кожного товару відображаються назва, зображення, ціна та кнопка переходу до детального перегляду.

Каталог забезпечує зручний пошук та перегляд товарів, дозволяючи користувачам швидко ознайомитися з доступним асортиментом. Інтерфейс сторінки побудований таким чином, щоб забезпечити просту навігацію та швидкий доступ до основних функцій інтернет-магазину (див. рис. 4.2).

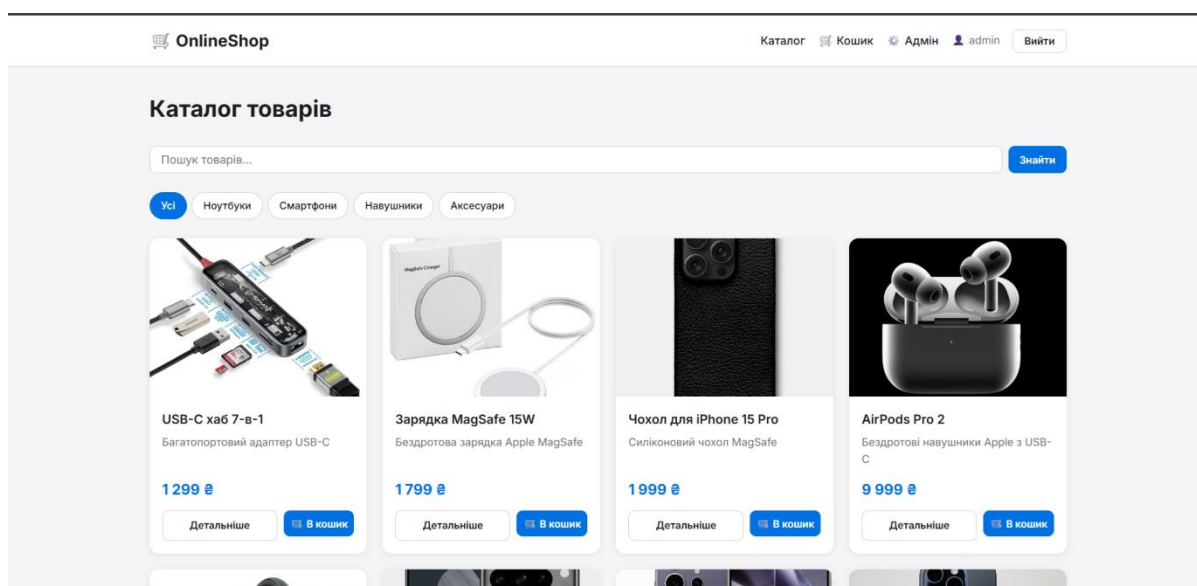


Рис. 4.2 Головна сторінка та каталог товарів вебзастосунку

4.3.2 Сторінка перегляду товару

Для кожного товару створена окрема сторінка, де можна детально ознайомитися з ним. На цій сторінці є вся інформація, яка потрібна, щоб вирішити, чи купувати товар. Тут можна побачити назву товару, його зображення, ціну, опис та інші характеристики.

Сторінка створюється автоматично, використовуючи дані з бази даних за номером вибраного товару. Це дозволяє користувачу завжди отримувати найсвіжішу інформацію про продукцію.

Реалізація детального перегляду робить вибір товарів зручнішим і

інформативнішим, а також дозволяє швидко додати товар у кошик.

Нижче наведено скриншот з вебзастосунку, як виглядає сторінка з боку користувача(див. рис. 4.3).

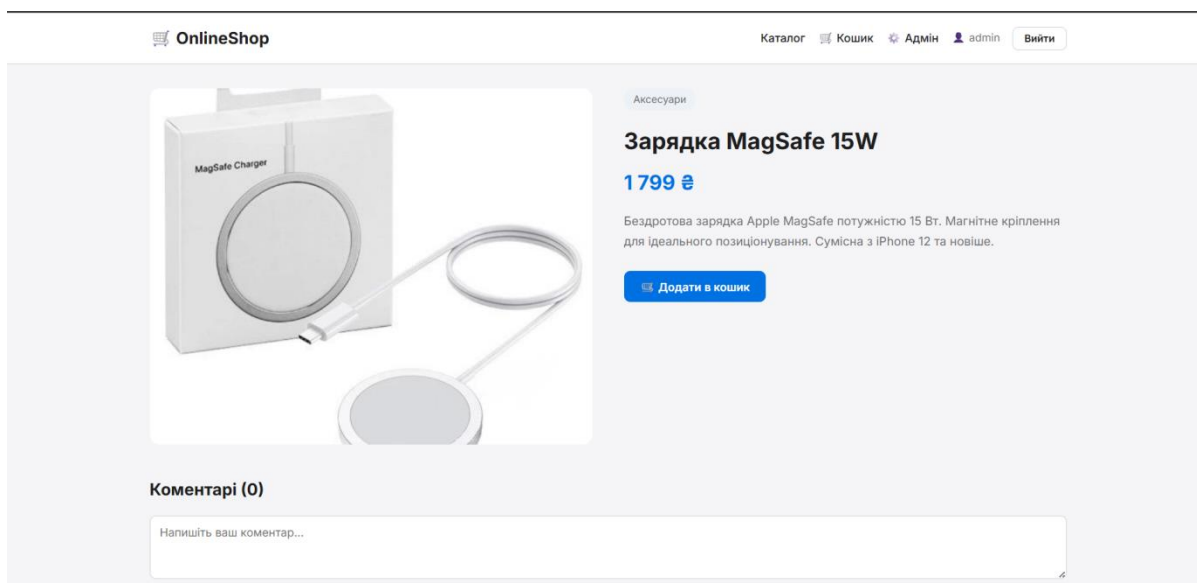


Рис. 4.3 Сторінка перегляду товару

4.3.3 Реалізація кошика покупця

Однією з головних функцій інтернет-магазину є можливість створення замовлення через кошик покупця. Для цього в вебзастосунку розроблено спеціальний механізм, який дає змогу користувачам додавати товари до кошика та управляти списком обраних товарів перед тим, як оформити покупку.

Додавати товари до кошика можна прямо зі сторінок каталогу та з детального перегляду товару. Коли ви натискаєте потрібну кнопку, інформація про вибраний товар зберігається в системі і стає доступною для подальшого перегляду. Такий спосіб дозволяє користувачу поступово формувати своє замовлення під час перегляду товарів магазину.

На сторінці кошика показується список усіх товарів, які були додані, з їх назвами, ціною та кількістю. Користувач може перевірити, чи все правильно в замовленні, подивитися загальну суму покупки і, якщо потрібно, видалити деякі товари з кошика. Це дає можливість зручно керувати замовленням і швидко вносити зміни перед оформленням.

Реалізація кошика покупця є важливою частиною вебзастосунку, оскільки саме через нього готуються товари до оформлення замовлення. Створена функція забезпечує зручність користування системою та відповідає основним вимогам сучасних інтернет-магазинів(див. рис. 4.4).

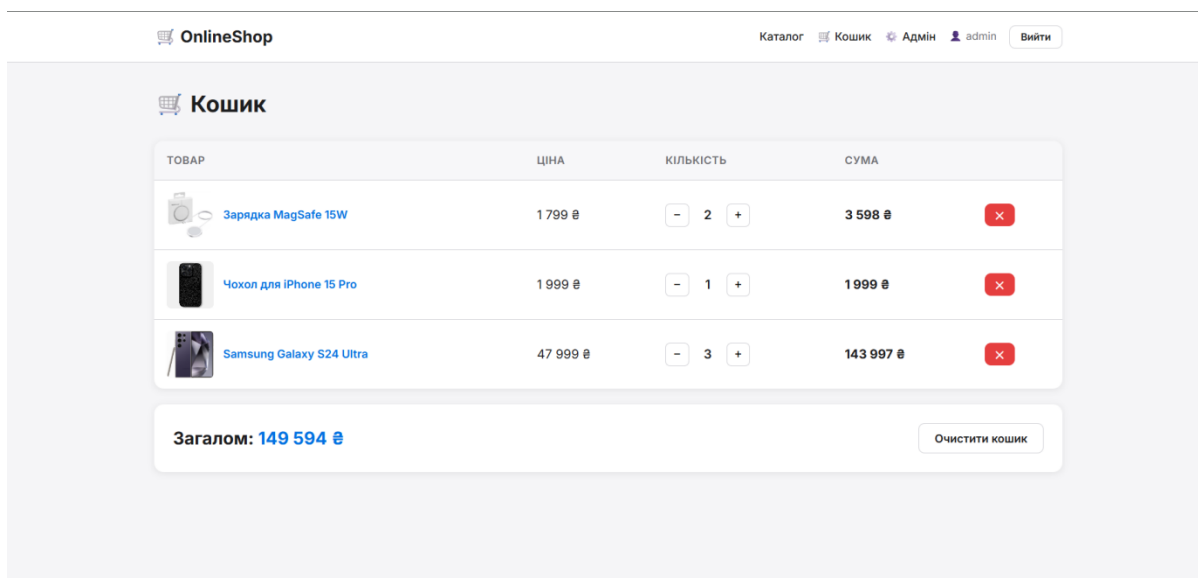


Рис. 4.4 Сторінка кошика покупця

Щоб отримати доступ до персоналізованих можливостей вебзастосунку, було впроваджено систему реєстрації та авторизації користувачів. Для створення облікового запису потрібно заповнити реєстраційну форму, де користувач вводить необхідні дані для подальшої роботи з системою.

Після того, як користувач успішно зареєструється, він може увійти в систему, використовуючи свої облікові дані. Коли він авторизується, сервер перевіряє, чи правильно введена інформація, і надає доступ до функцій в залежності від ролі користувача.

Система авторизації, яка була впроваджена, дозволяє обмежити доступ до певних функцій вебзастосунку для тих, хто не має авторизації, і забезпечує безпечну роботу з особистими даними та функціями інтернет-магазину (див. рис. 4.5, 4.6).

The screenshot shows the login page of the OnlineShop website. At the top left is the 'OnlineShop' logo with a shopping cart icon. At the top right are links for 'Каталог', 'Увійти', and 'Реєстрація'. The main content area features a white login form titled 'Вхід'. It contains two input fields: 'Email' with the value 'admin@shop.com' and 'Пароль' with masked characters '*****'. Below these is a blue 'Увійти' button. At the bottom of the form is a link: 'Немає акаунту? Зареєструватися'.

Рис. 4.5 Сторінка для входу користувача

The screenshot shows the registration page of the OnlineShop website. At the top left is the 'OnlineShop' logo with a shopping cart icon. At the top right are links for 'Каталог', 'Увійти', and 'Реєстрація'. The main content area features a white registration form titled 'Реєстрація'. It contains four input fields: 'Ім'я користувача' with the value 'admin@shop.com', 'Email' with the value 'your@email.com', 'Пароль' with masked characters '*****', and 'Підтвердження пароля' with the placeholder 'Повторіть пароль'. Below these is a blue 'Зареєструватися' button. At the bottom of the form is a link: 'Вже маєте акаунт? Увійти'.

Рис. 4.6 Сторінка для реєстрації користувача

4.3.4. Адміністративна панель керування

Для управління вмістом вебзастосунку була створена адміністративна панель. Вона призначена для системних адміністраторів і дає можливість користуватися функціями управління товарами, категоріями та іншими даними, які використовуються в роботі інтернет-магазину.

Адміністративна панель дає змогу додавати нові товари, редагувати вже існуючі записи та видаляти стару інформацію. Усі зміни виконуються через

спеціальні форми для введення даних і автоматично зберігаються в базі даних. Це значно полегшує процес оновлення інформації на сайті і не вимагає змінювати сам програмний код.

Інтерфейс адміністративної панелі створений так, щоб користувачі могли швидко отримувати доступ до основних функцій управління системою. Наявність окремої частини для адміністраторів робить підтримку вебзастосунку більш зручною і допомагає ефективно оновлювати інформацію, яка показується користувачам (див. рис. 4.7).

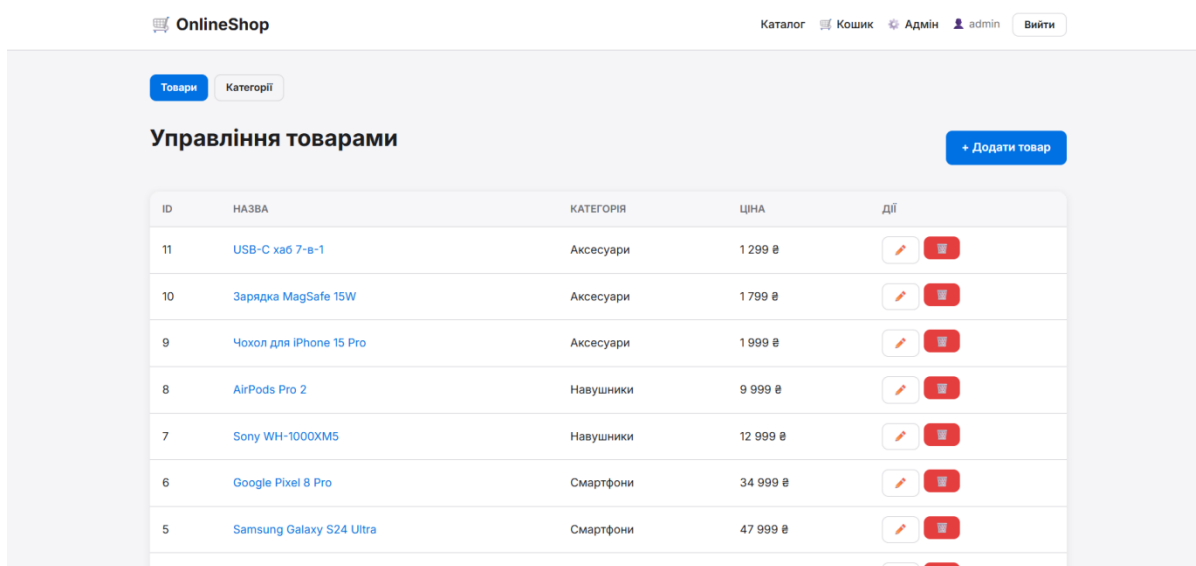


Рис. 4.7 Адміністративна панель керування

4.3.5 Створення замовлення користувачем

Однією з найважливіших можливостей розробленого вебзастосунку є оформлення замовлення користувачем. Даний функціонал дозволяє завершити процес покупки після вибору необхідних товарів та їх додавання до кошика.

Після переходу до кошика користувач може переглянути перелік вибраних товарів, їх кількість та загальну вартість замовлення. Якщо інформація є правильною, користувач підтверджує покупку, після чого система автоматично формує нове замовлення та зберігає його у базі даних.

Під час створення замовлення виконується перевірка наявності товарів у

кошику та обробка введених даних. Після успішного завершення операції інформація про замовлення додається до системи, а користувач отримує повідомлення про успішне оформлення покупки. Це дозволяє забезпечити зручний процес взаємодії із вебзастосунком та спрощує виконання основних дій під час роботи з інтернет-магазином.

Реалізація даного функціоналу є важливою частиною програмного продукту, оскільки забезпечує завершення процесу купівлі товарів та дозволяє організувати подальший облік створених замовлень. На рисунку нижче наведено програмну реалізацію створення замовлення користувачем(див. рис. 4.8).

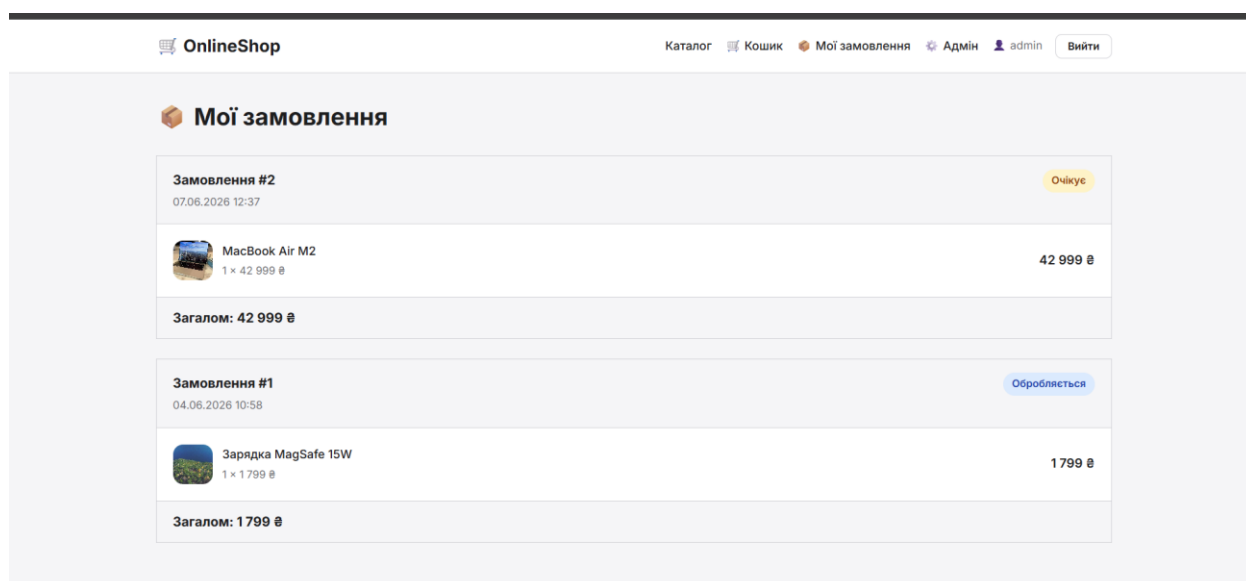


Рис. 4.8 Адміністративна панель керування

5 ТЕСТУВАННЯ ТА ПЕРЕВІРКА РОБОТИ СИСТЕМИ

5.1 Основи тестування програмного забезпечення

Тестування програмного забезпечення є важливим етапом у створенні будь-якої інформаційної системи, оскільки воно дозволяє перевірити, як правильно працюють функціональні модулі, виявити можливі помилки та оцінити, чи відповідає створений продукт вимогам технічного завдання. Головною метою тестування є забезпечення стабільної, надійної та передбачуваної роботи вебзастосунку в різних умовах використання.

У процесі створення інтернет-магазину тестування проводилося на різних рівнях: перевіряли окремі функції (наприклад, додавання товару до кошика), перевіряли, як компоненти системи взаємодіють між собою (клієнтська частина, серверна логіка та база даних), а також здійснювали комплексну перевірку основних сценаріїв використання системи кінцевим користувачем. Такий підхід допомагає виявити як локальні помилки, так і проблеми інтеграції між різними частинами застосунку.

Основним видом тестування в цьому проєкті було функціональне тестування, яке включає перевірку того, чи відповідає виконаний функціонал очікуваній поведінці системи. Зокрема, перевірялися правильність виконання операцій реєстрації та авторизації користувачів, відображення каталогу товарів, робота кошика покупця, а також функції адміністративної панелі.

Особливу увагу приділили тестуванню взаємодії з базою даних SQLite, адже вона відповідає за зберігання всієї інформації в системі. Перевіряли, чи правильно виконуються операції створення, читання, оновлення та видалення даних, що є дуже важливим для стабільної роботи інтернет-магазину.

Результати тестування показали, що всі основні функції вебзастосунку працюють правильно, а система стабільно обробляє запити користувачів і забезпечує правильне збереження та відображення даних.

5.2 Тестування основних сценаріїв роботи користувача

Для перевірки функціональності вебзастосунку було протестовано основні сценарії взаємодії користувача із системою. Перевірка включала реєстрацію нового користувача, авторизацію, перегляд каталогу товарів, перегляд детальної інформації про товар, додавання товару до кошика та роботу адміністративної панелі. Результати тестування наведені в таблиці 5.1.

Таблиця 5.1

Результати тестування основних сценаріїв роботи користувача

№	Тестовий сценарій	Очікуваний результат	Результат
1	Реєстрація користувача	Створення нового облікового запису	Виконано
2	Авторизація користувача	Успішний вхід до системи	Виконано
3	Перегляд каталогу товарів	Відображення списку товарів	Виконано
4	Перегляд інформації про товар	Відображення сторінки товару	Виконано
5	Додавання товару до кошика	Товар додано до кошика	Виконано
6	Робота адміністративної частини	Виконання операцій	Виконано

Результати проведеного тестування підтвердили коректність роботи основних функціональних можливостей вебзастосунку та відповідність отриманих результатів очікуваній поведінці системи.

5.3. Перевірка роботи бази даних

Під час тестування програмного забезпечення була проведена ретельна перевірка роботи бази даних SQLite, яка використовується як головне місце для зберігання даних вебзастосунку. База даних є важливою частиною системи, оскільки вона відповідає за збереження, обробку та відновлення інформації, яка потрібна для нормального функціонування всіх модулів.

Основна мета перевірки полягала в тому, щоб підтвердити правильність створення, зчитування та оновлення даних, а також оцінити стабільність роботи між серверною частиною програми. Особливу увагу приділили тому, щоб усі запити до бази даних виконувалися правильно, без втрат або спотворень інформації.

Під час тестування було перевірено, як працюють основні таблиці бази даних, що зберігають інформацію про користувачів, товари, категорії та елементи кошика.

Додатково була перевірена цілісність зв'язків між таблицями, зокрема, як працюють зовнішні ключі. Це дозволило впевнитись, що при зміні або видаленні пов'язаних записів не виникають помилки і структура бази даних залишається узгодженою. Також були протестовані ситуації, пов'язані з повторним додаванням даних і обмеженнями унікальності, що підтвердило правильну роботу правил цілісності.

Було окремо перевірено, як добре виконуються основні запити до бази даних. Результати показали, що операції вибірки, фільтрації та оновлення даних проходять швидко, без значних затримок, що дозволяє швидко бачити зміни у вебінтерфейсі застосунку.

За результатами тестування встановлено, що база даних SQLite працює стабільно, всі операції виконуються правильно, а взаємодія між базою даних і сервером є надійною та узгодженою (див. рис. 5.1 та 5.2).

	<u>Id</u>	<u>UserName</u>	<u>Email</u>	<u>PasswordHash</u>	<u>IsAdmin</u>	<u>CreatedAt</u>
	Ф...	Фильтр	Фильтр	Фильтр	Фильтр	Фильтр
1	1	admin	admin@shop.com	O2EsdaelBIpDX7bsgeUv+S1teVqLWpwXBw9q...	1	2026-05-25 07:42:48.1544902
2	2	Олександр	sherenok12@gmail.com	9jMkQ17TIQrTxPYiJutTGUoI5+I/...	0	2026-05-25 10:03:56.6201278

Рис. 5.1 Вигляд бази даних при сторенні користувача

Таблиця: Products

	ID	Name	ShortDescription	FullDescription	Price	
	Ф...	Фильтр	Фильтр	Фильтр	Фильтр	Фильтр
1	1	MacBook Air M2	Легкий та потужний ноутбук Apple	Apple MacBook Air з чіпом M2 ...	42999	https://raw
2	2	Lenovo ThinkPad X1 Carbon	Бізнес-ноутбук преміум класу	Lenovo ThinkPad X1 Carbon Gen 11 – ...	55999	https://git
3	3	ASUS ROG Strix G16	Ігровий ноутбук з RTX 4060	ASUS ROG Strix G16 – потужний ігрови...	47500	https://git
4	4	iPhone 15 Pro	флагманський смартфон Apple	iPhone 15 Pro з титановим корпусом, ...	49999	https://git
5	5	Samsung Galaxy S24 Ultra	флагман Samsung зі стилусом S Pen	Samsung Galaxy S24 Ultra з процесоро...	47999	https://git
6	6	Google Pixel 8 Pro	Смартфон Google з найкращою камерою	Google Pixel 8 Pro з чіпом Tensor G3...	34999	https://git
7	7	Sony WH-1000XM5	Бездротові навушники з ...	Sony WH-1000XM5 – преміальні ...	12999	https://raw
8	8	AirPods Pro 2	Бездротові навушники Apple з USB-C	AirPods Pro 2 з чіпом H2, адаптивним...	9999	https://git
9	9	Чохол для iPhone 15 Pro	Силиконовий чохол MagSafe	Оригінальний силиконовий чохол Apple...	1999	https://git
10	10	Зарядка MagSafe 15W	Бездротова зарядка Apple MagSafe	Бездротова зарядка Apple MagSafe ...	1799	https://git
11	11	USB-C хаб 7-в-1	Багатоportовий адаптер USB-C	USB-C хаб 7-в-1: HDMI 4K, 2x USB 3.0...	1299	https://git

Режим 1

Edit Type

Уда

ID :

Рис. 5.2 Вигляд бази даних продуктів

5.4. Результати тестування системи

Під час тестування перевірили стабільність і правильність роботи основних функцій вебдодатку. Система успішно виконує всі основні сценарії, включаючи взаємодію користувача з товарами, кошиком та адміністративними функціями.

Під час тестування ми особливо звертали увагу на те, як правильно виконуються основні операції в умовах реального використання (див. рис. 5.3).

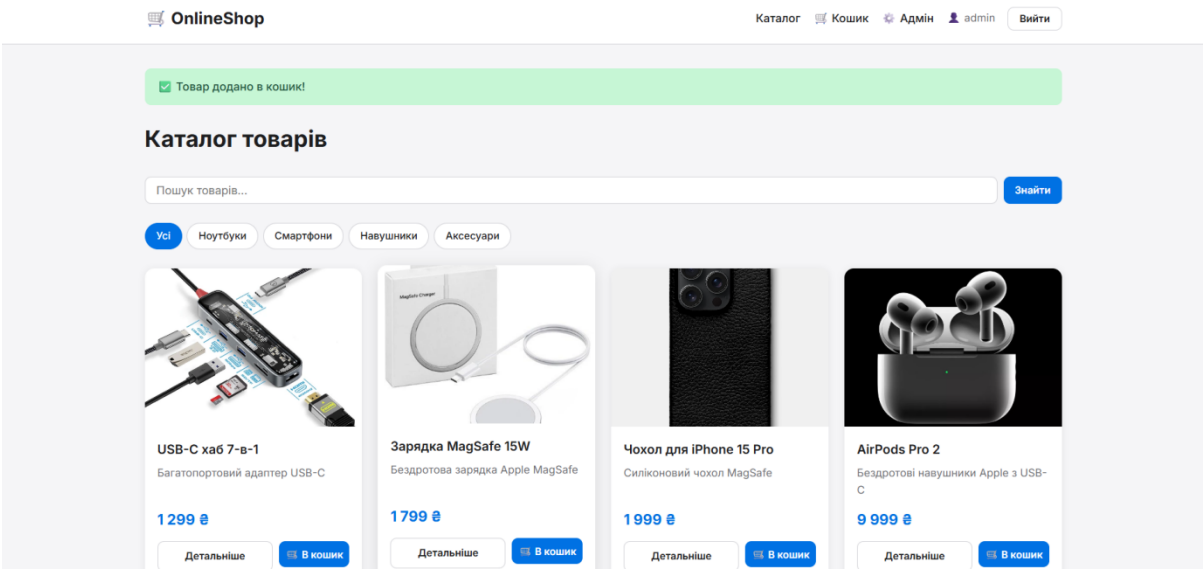


Рис. 5.3 Вигляд додавання товару в кошик

Було перевірено, як правильно працюють адміністративні функції, зокрема додавання, редагування і оформлення замовлень товарів у системі. Усі зміни одразу з'являються в базі даних і відображаються для користувача(див. рис. 5.4).

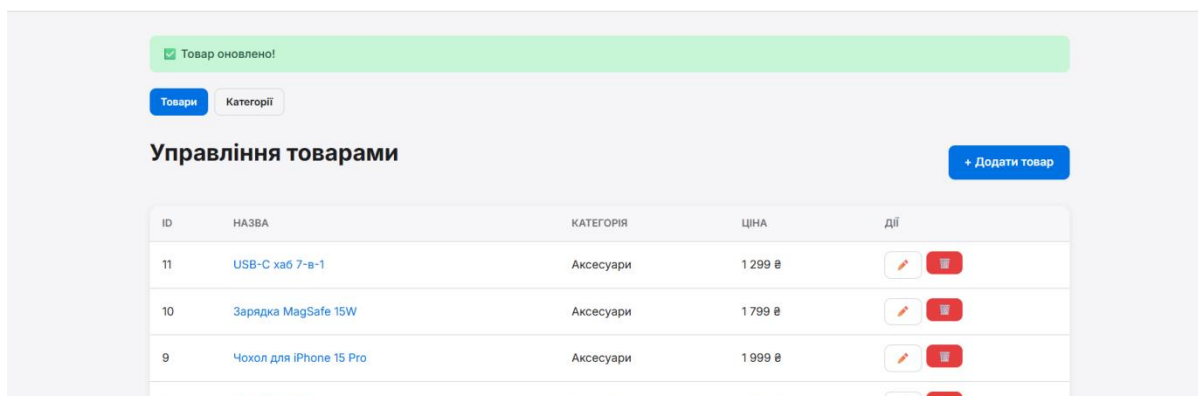


Рис. 5.4 Вигляд зміни даних товару

Отримані результати підтверджують, що вебзастосунок працює стабільно, а всі основні функції виконуються коректно без помилок.

ВИСНОВКИ

В результаті виконання курсового проєкту був створений вебзастосунок інтернет-магазину, який дозволяє користувачам повноцінно взаємодіяти з системою. Це включає перегляд каталогу товарів, отримання детальної інформації про продукти, додавання товарів до кошика, а також управління вмістом сайту адміністраторами. Розроблена система відповідає основним вимогам сучасних вебзастосунків і пропонує зручний та зрозумілий інтерфейс для користувачів.

Під час роботи над проєктом був проведений аналіз теми, що допомогло визначити основні функціональні вимоги до системи та структуру майбутнього вебзастосунку. На основі цього була розроблена архітектура програмного забезпечення, яка забезпечує чітке розділення логіки, відображення та обробки даних.

Реалізація серверної частини була зроблена за допомогою мови програмування C#. Для роботи з даними використали технологію Entity Framework Core, яка дозволяє легко взаємодіяти з базою даних SQLite. Це значно спростило процес розробки і підвищило гнучкість системи.

Користувацький інтерфейс був створений за допомогою HTML, CSS та JavaScript, що дало можливість зробити адаптивний і зручний дизайн вебзастосунку. Особливу увагу було приділено простоті навігації, логічному розташуванню елементів та забезпеченню комфортного спілкування користувача з системою.

У процесі розробки також був створений механізм авторизації користувачів, що забезпечує безпечний доступ до функцій системи, а також розподіл прав між користувачами та адміністратором. Окремо був реалізований функціонал кошика покупця, який дозволяє створювати замовлення та управляти вибраними товарами.

Тестування системи підтвердило, що всі основні модулі працюють правильно. Ми перевірили, як працює база даних, як взаємодіють серверна і клієнтська частини, а також основні способи використання системи. Результати показали, що вебзастосунок працює стабільно і виконує всі заплановані функції без помилок.

Таким чином, всі завдання, які були поставлені в рамках курсового проєкту,

були повністю виконані. Розроблена система може служити основою для подальшого розширення її можливостей, наприклад, шляхом додавання функції онлайн-оплати, більш детальної аналітики або інтеграції з зовнішніми сервісами. Проєкт показав практичні навички в розробці вебзастосунків за допомогою сучасних технологій та підходів у програмній інженерії.

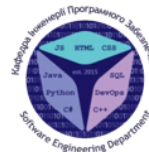
ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Freeman A. Pro ASP.NET Core 8: Develop Cloud-Ready Web Applications Using MVC, Blazor, and Razor Pages. 11th ed. New York : Apress, 2024. 1200 p.
2. Price M. J. C# 12 and .NET 8 – Modern Cross-Platform Development Fundamentals. 8th ed. Birmingham : Packt Publishing, 2024. 850 p.
3. Troelsen A., Japikse P. Pro C# 10 with .NET 6: Foundational Principles and Practices in Programming. New York : Apress, 2022. 1100 p.
4. Офіційна документація Microsoft з ASP.NET Core [Електронний ресурс]. – Режим доступу: <https://learn.microsoft.com/uk-ua/aspnet/core/>
5. Офіційна документація Entity Framework Core [Електронний ресурс]. – Режим доступу: <https://learn.microsoft.com/uk-ua/ef/core/>
6. Офіційна документація SQLite [Електронний ресурс]. – Режим доступу: <https://www.sqlite.org/docs.html>
7. Документація з мови програмування C# [Електронний ресурс]. –Режим доступу: <https://learn.microsoft.com/uk-ua/dotnet/csharp/>
- 8.Офіційна документація платформи .NET [Електронний ресурс]. – Режим доступу: <https://learn.microsoft.com/uk-ua/dotnet/>
9. Документація ASP.NET Core MVC [Електронний ресурс]. – Режим доступу: <https://learn.microsoft.com/uk-ua/aspnet/core/mvc/>
10. Документація HTML5 від Mozilla Developer Network (MDN) [Електронний ресурс]. – Режим доступу: <https://developer.mozilla.org/en-US/docs/Web/HTML>
11. Документація CSS3 від Mozilla Developer Network (MDN) [Електронний ресурс]. – Режим доступу: <https://developer.mozilla.org/en-US/docs/Web/CSS>
12. Документація JavaScript від Mozilla Developer Network (MDN) [Електронний ресурс]. – Режим доступу: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>

ДОДАТОК А. ДЕМОНСТРАЦІЙНІ МАТЕРІАЛИ



ДЕРЖАВНИЙ УНІВЕРСИТЕТ
ІНФОРМАЦІЙНО-КОМУНІКАЦІЙНИХ ТЕХНОЛОГІЙ
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ ІНФОРМАЦІЙНИХ
ТЕХНОЛОГІЙ



Кафедра Інженерії програмного забезпечення

Розробка програмного забезпечення для керування базою даних магазину

Виконав:

Студент групи ПД-22 Шеренок Олександр Олександрович

Керівник:

Ярошевський Олександр Вікторович
асистент кафедри Інженерії програмного забезпечення

Київ - 2026

МЕТА, ОБ'ЄКТА ТА ПРЕДМЕТ ДОСЛІДЖЕННЯ

Мета роботи — розробка функціонального інтернет-магазину на платформі ASP.NET Core, який забезпечує перегляд каталогу товарів, додавання їх до кошика, реєстрацію та авторизацію користувачів, а також керування контентом через адміністративну панель.

Об'єкт дослідження — процес розробки вебдодатку "OnlineShop" на платформі ASP.NET Core з використанням архітектурного патерну MVC.

Предмет дослідження — використання сучасних вебтехнологій, зокрема ASP.NET Core MVC, Entity Framework Core та бази даних SQLite, у процесі створення повноцінної клієнт-серверної системи інтернет-магазину.

АНАЛІЗ АНАЛОГІВ

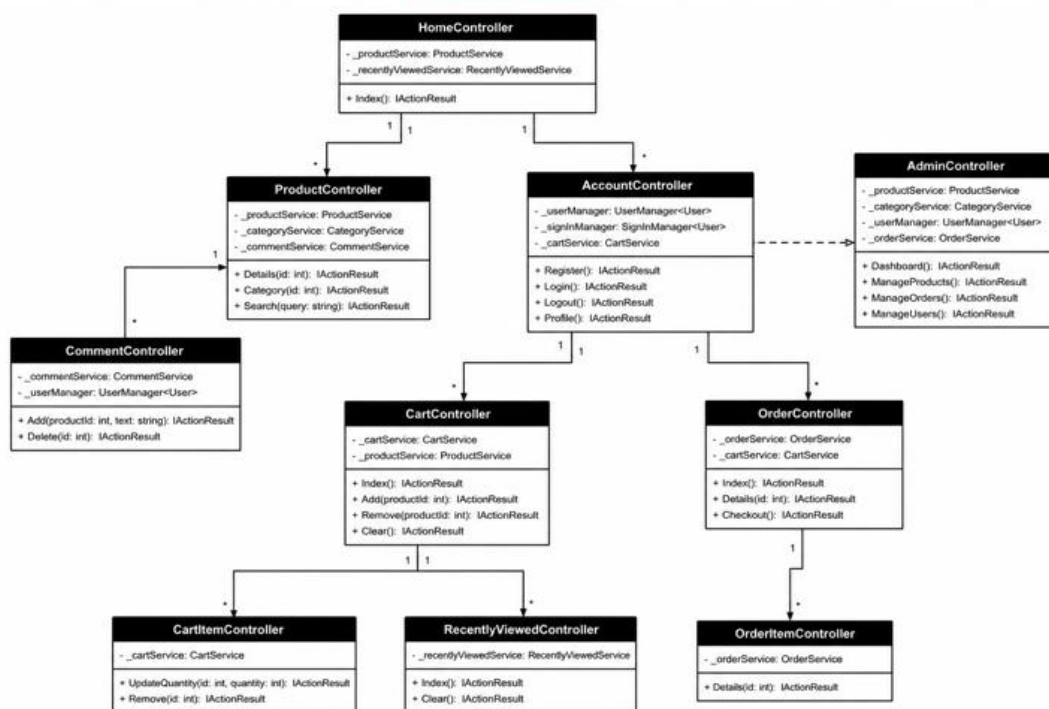
Назва	Amazon	Rozetka	OnlineShop
Тип застосунку	Веб / мобільний	Веб / мобільний	Веб
Архітектура	Мікросервісна	Модульна	MVC (N-рівнева)
Мова та фреймворк	Java / Python	PHP / Java	C#
База даних	DynamoDB / Aurora	PostgreSQL	SQLite
Авторизація	Повна (OAuth, 2FA)	Повна	Вбудована (Cookie Auth)
Пошук та фільтрація	Розширений	Розширений	Базовий (за категорією)
Кошик покупця	Так	Так	Так
Адмін-панель	Так	Так	Так
Валідація даних	Налаштована	Налаштована	Стандартна (Data Annotations)

ВИМОГИ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

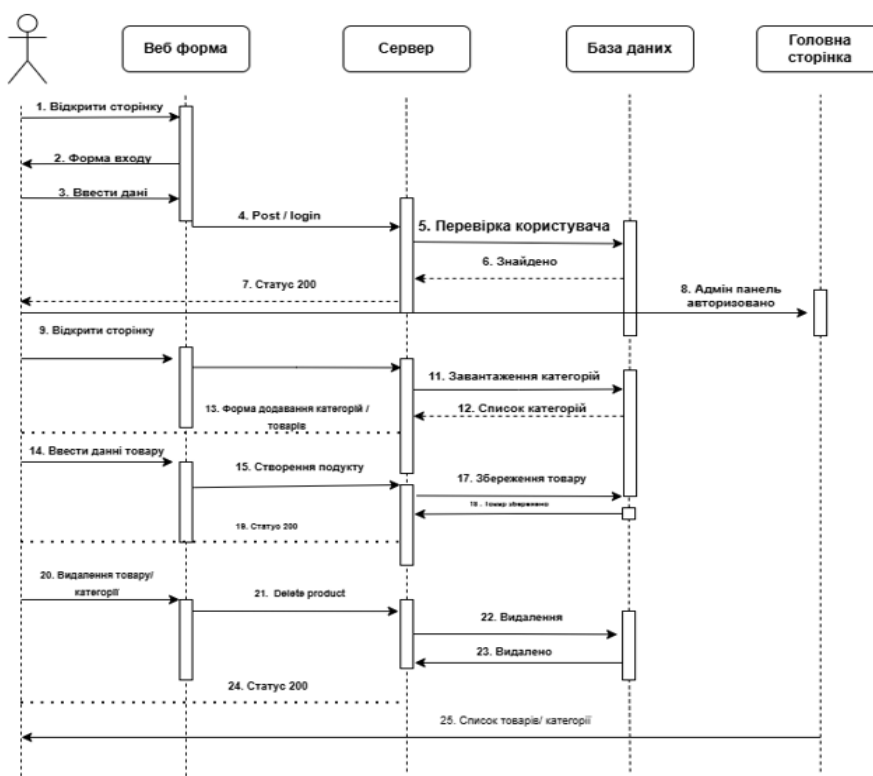
Функціональні вимоги: реєстрація та авторизація користувачів, перегляд каталогу товарів з пагінацією, пошук і фільтрація продукції за категорією та ціною, перегляд детальної інформації про товари, додавання та видалення товарів із кошика, оформлення замовлення з вибором способу доставки, перегляд історії замовлень у особистому кабінеті, можливість залишати коментарі до продукції, а також керування товарами для адміністратора (додавання, редагування, видалення).

Нефункціональні вимоги: час відповіді сервера не перевищує 2 секунди при навантаженні до 500 одночасних користувачів, коефіцієнт доступності системи — не менше 99,5% на місяць, паролі зберігаються у хешованому вигляді (bcrypt), передача даних здійснюється через HTTPS, інтерфейс відповідає принципам адаптивного дизайну та коректно відображається на екранах від 320px, архітектура допускає додавання нових модулів без зміни існуючого коду, система сумісна з браузерами Chrome, Firefox, Safari та Edge актуальних версій.

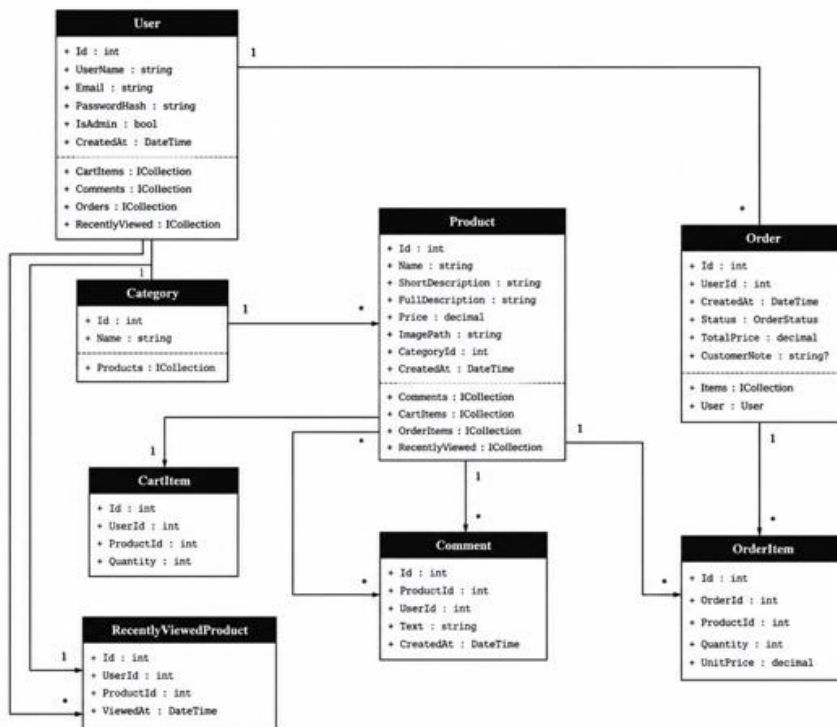
Діаграма класів контролерів вебзастосунку



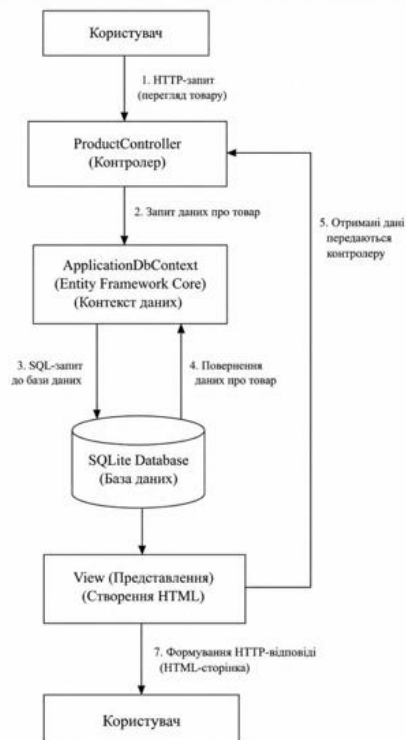
Діаграма послідовності



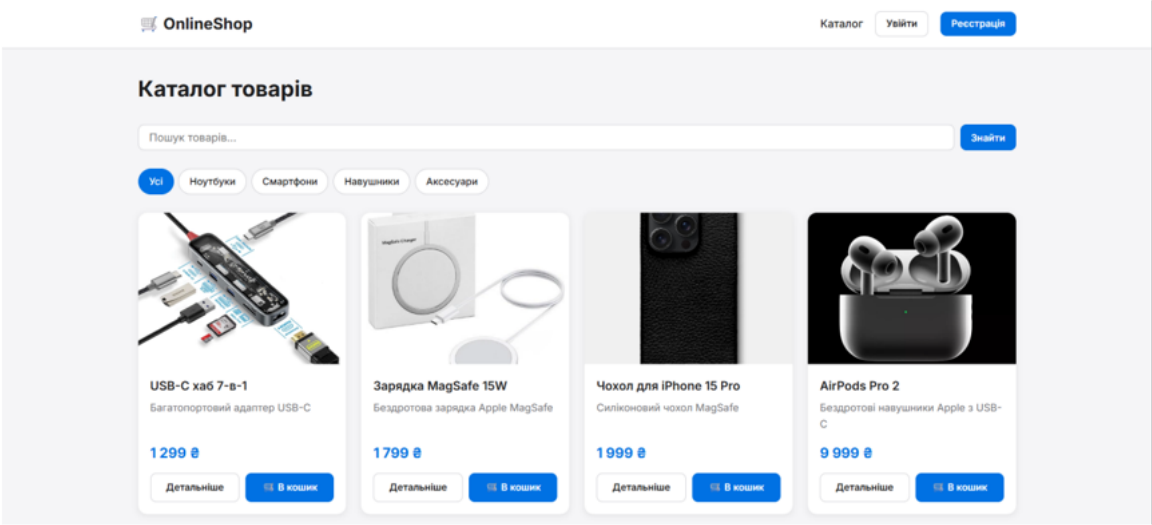
Діаграма класів баз даних



Діаграма обробки запиту на перегляд товару

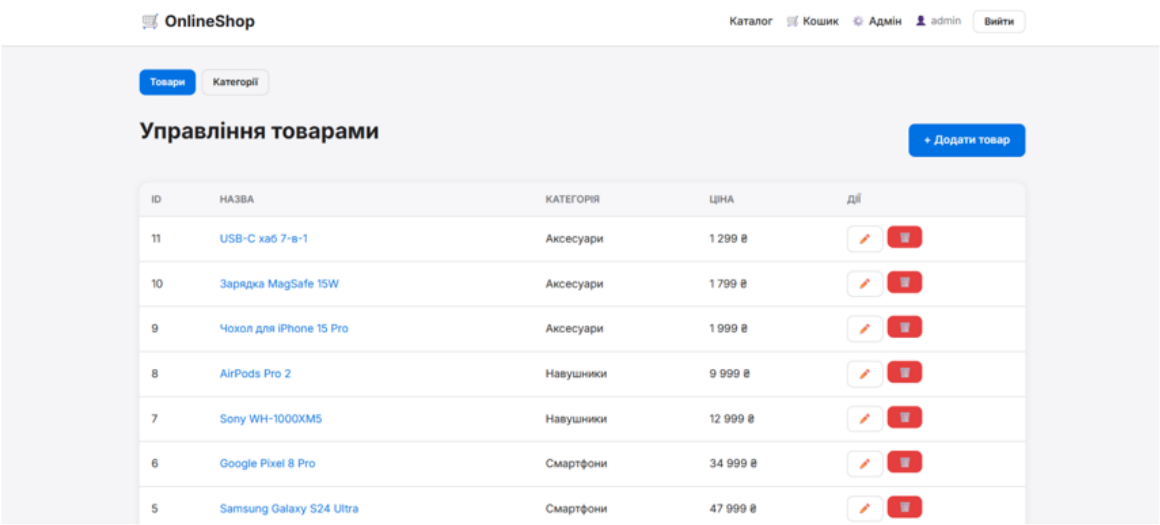


Головний екран вебзастосунку



11

Панель управління адміна



12

 OnlineShop

Каталог

Увійти

Регістрація

Каталог товарів

Пошук товарів...

Знайти

Усі

Ноутбуки

Смартфони

Наушники

Акcesуари



USB-C хаб 7-в-1
Багатопортовий адаптер USB-C

1 299 ₪

Детальніше

В кошик



Зарядка MagSafe 15W
Бездротова зарядка Apple MagSafe

1 799 ₪

Детальніше

В кошик



Чохол для iPhone 15 Pro
Силіконовий чохол MagSafe

1 999 ₪

Детальніше

В кошик



AirPods Pro 2
Бездротові навушники Apple з USB-C

9 999 ₪

Детальніше

В кошик

Головний екран входу

OnlineShop

Каталог Увійти **Регістрація**

Вхід

Email

your@email.com

Пароль

Увійти

Немає акаунту? Зареєструватися

10

Головний екран кошику

OnlineShop

Каталог Кошик Адмін admin **Вийти**

Кошик

ТОВАР	ЦІНА	КІЛЬКІСТЬ	СУМА
 Зарядка MagSafe 15W	1 799 ₪	- 3 +	5 397 ₪
 Чохол для iPhone 15 Pro	1 999 ₪	- 1 +	1 999 ₪
 Samsung Galaxy S24 Ultra	47 999 ₪	- 3 +	143 997 ₪

Загалом: 151 393 ₪

Очистити кошик

13

ВИСНОВКИ

У результаті виконання курсового проєкту було розроблено вебзастосунок інтернет-магазину на платформі ASP.NET Core з використанням мови програмування C# та бази даних SQLite.

Створена система дозволяє користувачам переглядати каталог товарів, здійснювати пошук і фільтрацію продукції, переглядати детальну інформацію про товари, реєструватися, авторизуватися та додавати товари до кошика. Для адміністратора реалізовано можливість керування товарами та даними системи.

Під час розробки було спроектовано структуру бази даних, реалізовано клієнтську та серверну частини застосунку, а також виконано тестування основних функцій. Результати тестування підтвердили коректну роботу системи та відповідність поставленим вимогам.

Усі поставлені завдання курсового проєкту були виконані, а розроблений вебзастосунок може бути основою для подальшого розвитку та розширення функціональних можливостей.

ДОДАТОК Б. РЕПОЗИТОРІЙ ПРОГРАМНИХ МОДУЛІВ

https://github.com/Sherenok12/Kursova_robota_Sherenok

ДОДАТОК В. ЛІСТИНГИ ОСНОВНИХ ПРОГРАМНИХ МОДУЛІВ

Модуль контролера облікових записів

<pre> public class AccountController : Controller { private readonly AppDbContext _db; public AccountController(AppDbContext db) => _db = db; [HttpGet] public IActionResult Login(string? returnUrl) { ViewBag.ReturnUrl = returnUrl; return View(); } [HttpPost] public async Task<IActionResult> Login(LoginViewModel model, string? returnUrl) { if (!ModelState.IsValid) return View(model); var hash = DbInitializer.HashPassword(model.Password); var user = await _db.Users.FirstOrDefaultAsync(u => u.Email == model.Email && u.PasswordHash == hash); if (user == null) { ModelState.AddModelError("", "Невірний email або пароль"); return View(model); } await SignInUser(user); return Redirect(returnUrl ?? "/"); } [HttpGet] public IActionResult Register() => View(); [HttpPost] public async Task<IActionResult> Register(RegisterViewModel model) </pre>	<pre> var user = new User { UserName = model.UserName, Email = model.Email, PasswordHash = DbInitializer.HashPassword(model.Password), CreatedAt = DateTime.UtcNow }; _db.Users.Add(user); await _db.SaveChangesAsync(); await SignInUser(user); return RedirectToAction("Index", "Home"); } [HttpPost] public async Task<IActionResult> Logout() { await HttpContext.SignOutAsync(CookieAuthenticationDefau lts.AuthenticationScheme); return RedirectToAction("Index", "Home"); } private async Task SignInUser(User user) { var claims = new List<Claim> { new(ClaimTypes.NameIdentifier, user.Id.ToString()), new(ClaimTypes.Name, user.UserName), new(ClaimTypes.Email, user.Email) }; </pre>
--	--

<pre>{ if (!ModelState.IsValid) return View(model); if (await _db.Users.AnyAsync(u => u.Email == model.Email)) { ModelState.AddModelError("Email", "Користувач з таким email вже існує"); return View(model); } }</pre>	<pre>if (user.IsAdmin) claims.Add(new Claim(ClaimTypes.Role, "Admin")); var identity = new ClaimsIdentity(claims, CookieAuthenticationDefaults.AuthenticationScheme); var principal = new ClaimsPrincipal(identity); await HttpContext.SignInAsync(CookieAuthenticationDefaults.AuthenticationScheme, principal, new AuthenticationProperties { IsPersistent = true }); }</pre>
---	---

Модуль контролера адміна

```

namespace OnlineShop.Controllers;
[Authorize(Roles = "Admin")]
public class AdminController : Controller
{
    private readonly AppDbContext _db;
    public AdminController(AppDbContext db) =>
        _db = db;
    // ===== Products =====
    public async Task<IActionResult> Products()
    {
        var products = await _db.Products.Include(p
=> p.Category).OrderByDescending(p =>
p.CreatedAt).ToListAsync();
        return View(products);
    } [HttpGet]
    public async Task<IActionResult>
CreateProduct()
    {
        var vm = new ProductFormViewModel {
Categories = await _db.Categories.ToListAsync() };
        return View(vm);
    }
    [HttpPost]
    public async Task<IActionResult>
CreateProduct(ProductFormViewModel model)
    {
        model.Categories = await
_db.Categories.ToListAsync();
        if (!ModelState.IsValid) return View(model);
        var product = new Product
        {
            Name = model.Name,
            ShortDescription = model.ShortDescription,
            FullDescription = model.FullDescription,
            Price = model.Price,
            ImagePath = model.ImagePath,
            CategoryId = model.CategoryId,
            CreatedAt = DateTime.UtcNow
        };
        if (product == null) return NotFound();
        product.Name = model.Name;
        product.ShortDescription =
model.ShortDescription;
        product.FullDescription = model.FullDescription;
        product.Price = model.Price;
        product.ImagePath = model.ImagePath;
        product.CategoryId = model.CategoryId;
        await _db.SaveChangesAsync();
        TempData["Message"] = "Товар оновлено!";
        return RedirectToAction("Products");
    } [HttpPost]
    public async Task<IActionResult> DeleteProduct(int
id)
    {
        var product = await _db.Products.FindAsync(id);
        if (product != null)
        {
            _db.Products.Remove(product);
            await _db.SaveChangesAsync();
            TempData["Message"] = "Товар видалено!";
        }
        return RedirectToAction("Products");
    }
    public async Task<IActionResult> Categories()
    {
        var categories = await _db.Categories.Include(c =>
c.Products).ToListAsync();
        return View(categories);
    }
    [HttpGet]
    public IActionResult CreateCategory() => View();
    [HttpPost]
    public async Task<IActionResult>
CreateCategory(Category model)
    {
        if (!ModelState.IsValid) return View(model);
        _db.Categories.Add(model);
        await _db.SaveChangesAsync();
        TempData["Message"] = "Категорію створено!";
        return RedirectToAction("Categories");
    } [HttpGet]

```

```

        _db.Products.Add(product);
        await _db.SaveChangesAsync();
        TempData["Message"] = "Товар створено!";
        return RedirectToAction("Products");
    }

    [HttpGet]
    public async Task<IActionResult>
    EditProduct(int id)
    {
        var product = await
        _db.Products.FindAsync(id);
        if (product == null) return NotFound();
        var vm = new ProductFormViewModel
        {
            Id = product.Id,
            Name = product.Name,
            ShortDescription =
product.ShortDescription,
            FullDescription = product.FullDescription,
            Price = product.Price,
            ImagePath = product.ImagePath,
            CategoryId = product.CategoryId,
            Categories = await
        _db.Categories.ToListAsync()
        };
        return View(vm);
    } [HttpPost]
    public async Task<IActionResult>
    EditProduct(ProductFormViewModel model)
    {
        model.Categories = await
        _db.Categories.ToListAsync();
        if (!ModelState.IsValid) return View(model);
        var product = await
        _db.Products.FindAsync(model.Id);
        public async Task<IActionResult> EditCategory(int
        id)
        {
            var category = await _db.Categories.FindAsync(id);
            if (category == null) return NotFound();
            return View(category);
        }

        [HttpPost]
        public async Task<IActionResult>
        EditCategory(Category model)
        {
            if (!ModelState.IsValid) return View(model);
            var category = await
            _db.Categories.FindAsync(model.Id);
            if (category == null) return NotFound();
            category.Name = model.Name;
            await _db.SaveChangesAsync();
            TempData["Message"] = "Категорію оновлено!";
            return RedirectToAction("Categories");
        } [HttpPost]
        public async Task<IActionResult> DeleteCategory(int
        id)
        {
            var category = await _db.Categories.Include(c =>
            c.Products).FirstOrDefaultAsync(c => c.Id == id);
            if (category != null) {
                if (category.Products.Any()) {
                    TempData["Error"] = "Неможливо видалити
категорію з товарами!";
                    return RedirectToAction("Categories");
                }
                _db.Categories.Remove(category);
                await _db.SaveChangesAsync();
                TempData["Message"] = "Категорію
видалено!";
            }
            return RedirectToAction("Categories");
        }
    }

```

Модуль CartController для керування кошиком покупця

```

namespace OnlineShop.Controllers;

[Authorize]
public class CartController : Controller
{
    private readonly ICartService _cart;

    public CartController(ICartService cart) => _cart
    = cart;

    private int GetUserId() =>
    int.Parse(User.FindFirst(ClaimTypes.NameIdentifier)!.Value);

    public async Task<IActionResult> Index()
    {
        var items = await
        _cart.GetCartItemsAsync(GetUserId());
        return View(new CartViewModel { Items =
        items });
    }

    [HttpPost]
    public async Task<IActionResult> Add(int
    productId, string? returnUrl)
    {
        await _cart.AddToCartAsync(GetUserId(),
        productId);

        TempData["Message"] = "Товар додано в кошик!";
        return Redirect(returnUrl ?? "/");
    }

    [HttpPost]
    public async Task<IActionResult> UpdateQuantity(int
    cartItemId, int quantity)
    {
        await _cart.UpdateQuantityAsync(cartItemId,
        quantity);
        return RedirectToAction("Index");
    }

    [HttpPost]
    public async Task<IActionResult> Remove(int
    cartItemId)
    {
        await _cart.RemoveFromCartAsync(cartItemId);
        return RedirectToAction("Index");
    }

    [HttpPost]
    public async Task<IActionResult> Clear()
    {
        await _cart.ClearCartAsync(GetUserId());
        return RedirectToAction("Index");
    }
}

```

Модуль, що реалізує перегляд товару, відгуків та рекомендацій

<pre> public class ProductController : Controller { private readonly AppDbContext _db; private readonly IRecentlyViewedService _recentlyViewed; public ProductController(AppDbContext db, IRecentlyViewedService recentlyViewed) { _db = db; _recentlyViewed = recentlyViewed; } private int? GetUserId() { var claim = User.FindFirst(ClaimTypes.NameIdentifier); return claim != null ? int.Parse(claim.Value) : null; } public async Task<IActionResult> Details(int id) { var product = await _db.Products .Include(p => p.Category) .FirstOrDefaultAsync(p => p.Id == id); if (product == null) return NotFound(); var comments = await _db.Comments .Include(c => c.User) .Where(c => c.ProductId == id) .OrderByDescending(c => c.CreatedAt) .ToListAsync(); </pre>	<pre> var similar = await _db.Products .Where(p => p.CategoryId == product.CategoryId && p.Id != product.Id) .Take(4) .ToListAsync(); var recentlyViewed = new List<Models.Product>(); var userId = GetUserId(); if (userId.HasValue) { await _recentlyViewed.AddViewAsync(userId.Value, id); recentlyViewed = await _recentlyViewed.GetRecentlyViewedAsync(userId.Valu e, id); } var vm = new ProductDetailViewModel { Product = product, Comments = comments, SimilarProducts = similar, RecentlyViewed = recentlyViewed }; return View(vm); } </pre>
--	---

Модуль для відображення каталогу товарів

<pre> public class HomeController : Controller { private readonly AppDbContext _db; public HomeController(AppDbContext db) => _db = db; public async Task<IActionResult> Index(int? categoryId, string? search) { var query = _db.Products.Include(p => p.Category).AsQueryable(); if (categoryId.HasValue) query = query.Where(p => p.CategoryId == categoryId.Value); </pre>	<pre> if (!string.IsNullOrEmpty(search)) query = query.Where(p => p.Name.Contains(search) p.ShortDescription.Contains(search)); var products = await query.OrderByDescending(p => p.CreatedAt).ToListAsync(); var categories = await _db.Categories.ToListAsync(); ViewBag.Categories = categories; ViewBag.SelectedCategory = categoryId; ViewBag.SearchQuery = search; return View(products); } } </pre>
--	--